



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

AUTOENKODERY NEUROSymboliczne dla modelowania sekwencyjnych struktur dwuwymiarowych

Paweł Chumski, 144392

Promotor
prof. dr hab. inż. Krzysztof Krawiec

POZNAŃ 2026

Podziękowania

Pragnę złożyć serdeczne podziękowania mojemu promotorowi, Panu prof. dr. hab. inż. Krzysztofowi Krawcowi, za cenne wskazówki merytoryczne oraz ogromną wyrozumiałość w trakcie powstawania niniejszej pracy.

Szczególne słowa wdzięczności kieruję do mojej dziewczyny i całej rodziny za niezachwiane wsparcie psychiczne, cierpliwość oraz motywację do działania w najtrudniejszych momentach.

Dziękuję również moim znajomym ze studiów – w szczególności dwóm wspaniałym kolegom i koleżance – za wzajemną pomoc, wymianę wiedzy i świetną atmosferę, która towarzyszyła nam przez cały okres nauki.

Streszczenie

Gwałtowny rozwój głębokich sieci neuronowych przyniósł znaczący postęp w analizie obrazu, jednak klasyczne modele wciąż zmagają się z problemem braku interpretowalności (tzw. czarna skrzynka) oraz trudnością w modelowaniu strukturalnych relacji między obiektami. W odpowiedzi na te wyzwania, niniejsza praca podejmuje tematykę sztucznej inteligencji neurosymbolicznej (ang. *neurosymbolic AI*), integrującej percepcję opartą na danych ze strukturalnym rozumowaniem.

Celem pracy było zaprojektowanie, implementacja i ewaluacja autoenkodera neurosymbolicznego, zdolnego do nienadzorowanej dekompozycji i rekonstrukcji dwuwymiarowych, sekwencyjnych struktur wizualnych o zmiennej długości oraz złożonej topologii.

Zaproponowana hybrydowa architektura rozwijana była dwuetapowo. W pierwszej fazie wdrożono sekwencyjny moduł wnioskowania oparty na sieciach rekurencyjnych (GRU), generujący parametry geometryczne. Następnie, w odpowiedzi na ograniczenia modeli rekurencyjnych przy skomplikowanych topologiach (skrzyżowania, bifurkacje), zaprojektowano nowatorski przestrzenny dekodér siatkowy (ang. *Grid Decoder*). Obydwa warianty zintegrowano ze splotowym modułem percepcyjnym (CNN) oraz w pełni różniczkowalnym rendererem 2D.

System optymalizuje wielokryterialną funkcję straty w paradygmacie analizy przez synteze (ang. *analysis-by-synthesis*), ucząc się obiektowych reprezentacji bez jawnego etykietowania. Dla zapewnienia stabilności procesu treningowego wdrożono techniki uczenia programowanego (ang. *curriculum learning*), wymuszania nauczyciela (ang. *teacher forcing*) oraz innowacyjną analityczną separację błędów topologii i koloru.

Ewaluację empiryczną przeprowadzono na autorskim, parametrycznym zbiorze danych syntetycznych. Wyniki eksperymentów wykazały, że model z powodzeniem rozwiązuje problem zmiennej liczebności obiektów (problem stopu). Co najważniejsze, architektura siatkowa całkowicie zniwelowała ograniczenia sekwencyjne, bezbłędnie modelując przecinające się trajektorie. Ponadto system wykazał znaczącą odporność na celowe degradacje obrazu wejściowego, takie jak ekstremalny szum sensorowy, rozmycie optyczne czy utrata pikseli. Praca dowodzi, że integracja głębokiego uczenia z renderowaniem różniczkowalnym pozwala na budowę systemów wysoce interpretowalnych i świadomych swoich ograniczeń związanych z przesunięciem dziedziny (ang. *Domain Shift*), otwierając drogę do analizy rzeczywistych obrazów medycznych.

Słowa kluczowe: sztuczna inteligencja neurosymboliczna, renderowanie różniczkowalne, autoenkodery, głębokie uczenie, sieci rekurencyjne, reprezentacje obiektocentryczne.

Abstract

The rapid development of deep neural networks has brought significant progress in image analysis; however, classical models still struggle with the lack of interpretability (the black-box problem) and difficulties in modeling structural relationships between objects. In response to these challenges, this thesis explores neurosymbolic AI, integrating data-driven perception with structural reasoning.

The aim of this work was to design, implement, and evaluate a neurosymbolic autoencoder capable of unsupervised decomposition and reconstruction of two-dimensional, variable-length sequential visual structures and complex topologies.

The proposed hybrid architecture was developed in two stages. In the first phase, a sequential reasoning module based on recurrent neural networks (GRU) generating geometric parameters was implemented. Subsequently, in response to the limitations of recurrent models when handling complex topologies (intersections, bifurcations), a novel spatial grid decoder was designed. Both variants were integrated with a convolutional perception module (CNN) and a fully differentiable 2D renderer.

The system optimizes an aggregated loss function within the analysis-by-synthesis paradigm, learning object-centric representations without explicit labels. To ensure the stability of the training process, curriculum learning, teacher forcing, and analytical separation of topology and color errors were implemented.

Empirical evaluation was conducted on a custom, parametric synthetic dataset. Experimental results demonstrated that the model successfully solves the variable object count problem (the halting problem). Most importantly, the grid architecture completely overcame the limitations of the recurrent model, flawlessly reconstructing intersecting trajectories. Furthermore, the architecture exhibited significant robustness to intentional input image degradations, such as extreme Gaussian noise, optical blur, and pixel dropout. This thesis proves that the integration of deep learning with differentiable rendering enables the development of highly interpretable systems resilient to Domain Shift phenomena, opening new possibilities for real-world medical image analysis.

Keywords: neurosymbolic AI, differentiable rendering, autoencoders, deep learning, recurrent neural networks, object-centric representations.

Spis treści

1	Wprowadzenie	1
1.1	Kontekst i motywacja	1
1.2	Problem i wyzwania badawcze	1
1.3	Cel pracy i teza	2
1.4	Proponowane podejście i wkład własny	2
1.5	Struktura pracy	3
2	Tło teoretyczne	4
2.1	Autoenkodery i reprezentacja ukryta	4
2.2	Modelowanie sekwencyjne i sieci rekurencyjne	5
2.3	Struktury kompozycyjne i problem wiązania	6
2.4	Renderowanie różniczkowalne	7
2.5	Mechanizmy uwagi	7
3	Powiązane prace	8
3.1	Autoenkodery i uczenie reprezentacji obiektocentrycznych	8
3.2	Modelowanie sekwencyjne i generowanie szkiców	9
3.3	Reprezentacje kompozycyjne i modele AIR	9
3.4	Renderowanie różniczkowalne	10
3.5	Podejścia neurosymboliczne	10
3.6	Przestrzenne reprezentacje kompozycyjne oparte na gęstych siatkach	11
4	Proponowana metoda	12
4.1	Generowanie syntetycznego zbioru danych	12
4.1.1	Algorytm generacji sekwencyjnej	12
4.1.2	Parametryzacja geometrii i wyglądu	14
4.1.3	Reprezentacja danych wejściowych i wyjściowych	14
4.1.4	Rozszerzenie generatora o topologie złożone i skrzyżowania	15
4.2	Ogólna architektura systemu	16
4.3	Moduł percepcyjny (Enkoder)	18
4.3.1	Architektura sieci	18
4.3.2	Przestrzeń ukryta	19
4.4	Moduł wnioskowania (Modeler)	19
4.4.1	Wybór architektury rekurencyjnej (GRU vs LSTM)	19
4.4.2	Mechanizm integracji kontekstu	20
4.4.3	Dedykowane głowice wyjściowe	20
4.5	Reprezentacja wyjścia i dynamika ruchu	20
4.6	Kryteria agregowane funkcji straty z maskowaniem sekwencji	20

4.7	Matematyczny model renderowania różniczkowalnego	21
4.8	Modelowanie zakłóceń i augmentacja danych	22
4.9	Środowisko implementacji i wykorzystane narzędzia	23
4.10	Alternatywna architektura – Przestrzenny Dekoder Siatkowy	24
4.10.1	Koncepcja dekompozycji siatkowej i struktura tensora	24
4.10.2	Wyuczalne parametry skalowania topologii	25
4.10.3	Rozwiązanie problemu zapadania się optymalizacji	25
4.10.4	Separacja wnioskowania o topologii i kolorze	26
5	Ewaluacja empiryczna	27
5.1	Szczegóły implementacji modelu i parametry treningu	27
5.2	Metodyka i wyniki eksperymentów	28
5.2.1	Eksperyment 1: Wpływ przycinania gradientów na stabilność uczenia	28
5.2.2	Eksperyment 2: Dynamika uczenia z wykorzystaniem wymuszania nauczyciela	29
5.2.3	Eksperyment 3: Rekonstrukcja i zliczanie obiektów	30
5.2.4	Eksperyment 4: Badanie odporności	30
5.2.5	Eksperyment 5: Analiza ilościowa – metryka IoU	33
5.2.6	Eksperyment 6: Badanie ablacyjne przestrzeni ukrytej	33
5.2.7	Eksperyment 7: Analiza złożoności obliczeniowej modelu	35
5.2.8	Eksperyment 8: Rekonstrukcja topologii złożonych przestrzennym modelem siatkowym	35
5.2.9	Eksperyment 9: Analiza wyuczalnych parametrów skalowania i benchmark architektur	39
5.3	Zestawienie porównawcze	39
5.4	Analiza ograniczeń i przypadków brzegowych	40
6	Wnioski i kierunki dalszych badań	43
6.1	Wyzwania implementacyjne i analiza stabilności	44
6.2	Ewolucja podejścia i nieudane ścieżki badawcze	44
6.2.1	Ewolucja podejścia do bifurkacji i ograniczenia modeli sekwencyjnych	44
6.2.2	Podejście oparte na strukturach kolejkowych i BFS	44
6.3	Kierunki dalszych badań	45
6.4	Autorefleksja metodologiczna i zagrożenia	45
	Literatura	47
	Dodatek A: Kod źródłowy i odtwarzanie eksperymentów	50

Rozdział 1

Wprowadzenie

1.1 Kontekst i motywacja

W ostatnich latach rozwój metod głębokiego uczenia maszynowego (ang. *deep learning*) znacząco przyczynił się do postępu w przetwarzaniu i analizie danych wizualnych [1]. Klasyczne splotowe sieci neuronowe (ang. *CNN*) wykazały wyjątkową skuteczność w zadaniach klasyfikacji czy detekcji obiektów. Jednakże, mimo wysokiej skuteczności predykcyjnej, ich fundamentalnym ograniczeniem pozostaje natura „czarnej skrzynki” (ang. *black box*) – trudność w interpretacji mechanizmów decyzyjnych oraz brak jawnej, strukturalnej reprezentacji przetwarzanych danych [2].

W odpowiedzi na te niedoskonałości, coraz większą uwagę badaczy przyciągają podejścia **neurosymboliczne** (ang. *neurosymbolic AI*). Paradygmat ten dąży do unifikacji dwóch historycznie rozłącznych nurtów sztucznej inteligencji: koneksjonizmu (ang. *connectionism*), oferującego potężne zdolności uczenia się z surowych danych, oraz symbolizmu, zapewniającego zdolność do logicznego rozumowania i interpretowalności [3]. Celem systemów neurosymbolicznych jest ekstrakcja z danych wizualnych takich reprezentacji, które są nie tylko wektorami liczbowymi, ale posiadają strukturę semantyczną (np. lista obiektów, ich atrybuty i relacje) [4]. Warto podkreślić, że systemy neurosymboliczne stanowią ogólny paradygmat integracji logiki i głębokiego uczenia, nieograniczający się wyłącznie do przetwarzania danych wizualnych, choć w niniejszej pracy skupiono się na tym właśnie aspekcie.

Równolegle, techniki **renderowania różniczkowalnego** (ang. *differentiable rendering*) otwierają nowe możliwości w uczeniu reprezentacji metodą „analizy przez syntezę” (ang. *analysis-by-synthesis*). Podejście to zakłada, że model powinien umieć nie tylko rozpoznać obraz, ale także go zrekonstruować, wykorzystując w tym celu zdefiniowany zestaw prostych reguł i prymitywów geometrycznych [5, 6]. Pozwala to na dekompozycję złożonej sceny na interpretowalne komponenty, zachowując spójność semantyczną i umożliwiając ich późniejszą manipulację.

1.2 Problem i wyzwania badawcze

Jednym z fundamentalnych wyzwań pozostaje problem wiązania cech (ang. *binding problem*), pojęcie wywodzące się z psychologii poznawczej, a adaptowane w uczeniu maszynowym jako problem uczenia się hierarchicznych, kompozycyjnych reprezentacji w trybie nienadzorowanym [7].

Szczególnie wymagającym przypadkiem są dane o strukturze sekwencyjnej i zmiennej złożoności, gdzie liczba obiektów na scenie nie jest z góry ustalona. Wymaga to od modelu jednoczesnego:

- lokalizacji obiektów w przestrzeni ciągłej (współrzędne x, y),

- określenia ich atrybutów (kolor, rozmiar),
- modelowania relacji strukturalnych (kolejność elementów, np. w rysunku odręcznym),
- decydowania o momencie zakończenia generowania (problem stopu).

Przykłady takich struktur można znaleźć w wielu dziedzinach, takich jak: rozpoznawanie szkiców technicznych [8], analiza trajektorii ruchu [9], czy modelowanie struktur biologicznych w obrazach medycznych, gdzie struktury często przybierają formę łańcuchów lub skupisk o zmiennej liczebności [10]. Klasyczne podejścia sekwencyjne zawodzą w szczególności w momencie pojawiania się rozwidlenia i skrzyżowań trajektorii.

1.3 Cel pracy i teza

Głównym celem niniejszej pracy jest opracowanie, implementacja i ewaluacja architektury **auto-enkodera neurosymbolicznego**, zdolnego do rekonstrukcji sekwencyjnych struktur wizualnych o zmiennej długości oraz złożonej topologii. Jako tezę pracy przyjmuje się, że:

Zastosowanie hybrydowej architektury łączącej splotowy enkoder wizualny z dekodern strukturalnym (w wariancie opartym na sieciach rekurencyjnych oraz wariancie przestrzennym opartym na gęstej siatce), w połączeniu z renderowaniem różniczkowalnym, pozwala na uzyskanie interpretowalnej i manipulowalnej reprezentacji danych wizualnych bez konieczności stosowania jawnego nadzoru (etykietowania) poszczególnych komponentów.

Aby zweryfikować tę tezę, sformułowano następujące pytania badawcze:

- **P1:** W jakim stopniu autorska funkcja straty, uwzględniająca więzy geometryczne, pozwala na wymuszenie fizycznej spójności i ciągłości generowanych struktur?
- **P2:** Jakie mechanizmy modelowania (np. jednostki GRU, mechanizmy uwagi, przestrzenna dekompozycja siatkowa) są niezbędne do stabilnej rekonstrukcji struktur o zmiennej liczbie elementów i przecinających się trajektoriach?
- **P3:** Czy uzyskana w procesie uczenia nienadzorowanego reprezentacja ukryta pozwala na skuteczną manipulację atrybutami obiektów poprzez bezpośrednią ingerencję w parametry symboliczne?
- **P4:** W jaki sposób optymalizacja *end-to-end* globalnych parametrów skalujących (zamiast sztywnych hiperparametrów) wpływa na formowanie topologii w przestrzennych modelach siatkowych?

1.4 Proponowane podejście i wkład własny

W ramach pracy zaproponowano autorską architekturę modelu, składającą się z trzech zintegrowanych modułów:

1. **Enkoder** (ang. *vision module*) – oparty na splotowej sieci neuronowej (CNN), odpowiedzialny za ekstrakcję globalnego kontekstu wizualnego z obrazu wejściowego.
2. **Moduł wnioskowania** (ang. *symbolic reasoner*) – zrealizowany w dwóch wariantach. Pierwszy to moduł oparty na sieciach rekurencyjnych (GRU/LSTM), który sekwencyjnie generuje parametry opisujące prymitywy geometryczne (ich położenie, wymiary i wygląd). Drugi,

zaprojektowany jako odpowiedź na trudności w modelowaniu rozgałęzień, to wariant przestrzenny działający na gęstej siatce (ang. *grid-based spatial decoder*), realizujący równoległe wnioskowanie.

3. **Renderer różniczkowalny** – nienauczalny moduł, który przekształca wygenerowane parametry z powrotem w przestrzeń pikseli, umożliwiając obliczenie błędu rekonstrukcji i propagację gradientu przez cały system [11].

Głównym wkładem (ang. *contribution*) niniejszej pracy jest:

- Opracowanie metody stabilnego treningu modelu neurosymbolicznego z wykorzystaniem technik uczenia programowanego (ang. *curriculum learning*) [12] oraz wymuszania nauczyciela (ang. *teacher forcing*) [13].
- Wprowadzenie nowatorskiej architektury siatkowej (16×16) z globalnymi, wyuczalnymi parametrami skalowania promieni, optymalizowanymi analitycznie w procesie wstecznej propagacji błędu.
- Zaproponowanie złożonej funkcji celu (ang. *loss function*), uwzględniającej aspekty fizyczne (kolizje, ciągłość) oraz semantyczne (liczba obiektów), a także zapobiegającej załamaniom optymalizacji (ang. *Zero-Collapse*) przy pełnej zgodności przestrzeni barw (RGB).
- Przeprowadzenie eksperymentów wykazujących zdolność modelu do generalizacji na sekwencje o długościach niewidzianych podczas treningu oraz na złożone warianty symulujące ekstremalny szum sensorowy.

1.5 Struktura pracy

Układ pracy został zaplanowany w następujący sposób:

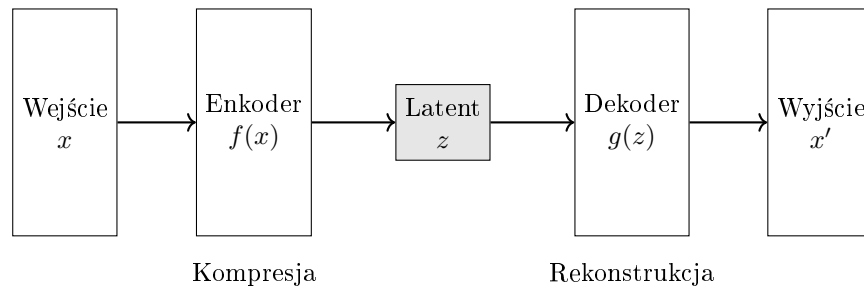
- **Rozdział 2: Tło teoretyczne** — przedstawia fundamenty głębokiego uczenia, ze szczególnym uwzględnieniem autoenkoderów, sieci rekurencyjnych oraz paradygmatu neurosymbolicznego.
- **Rozdział 3: Przegląd literatury** — omawia aktualny stan wiedzy w zakresie dekompozycji scen (ang. *scene decomposition*), obiektowych reprezentacji (ang. *object-centric learning*), renderowania różniczkowalnego oraz modeli operujących na gęstych siatkach.
- **Rozdział 4: Proponowana metoda** — zawiera szczegółowy opis techniczny zaprojektowanej architektury, ewolucję od modelu rekurencyjnego do modelu siatkowego oraz matematyczne sformułowanie renderera różniczkowalnego.
- **Rozdział 5: Ewaluacja empiryczna** — opisuje środowisko eksperymentalne, zbiory danych (w tym symulacje szumu USG) oraz prezentuje wyniki ilościowe i jakościowe rekonstrukcji topologii złożonych.
- **Rozdział 6: Podsumowanie** — syntetyzuje wnioski płynące z badań oraz zarysowuje perspektywy dalszego rozwoju zaprezentowanej metody m.in. w kontekście transferu wiedzy (ang. *transfer learning*) na rzeczywiste obrazy medyczne.

Rozdział 2

Tło teoretyczne

2.1 Autoenkodery i reprezentacja ukryta

Autoenkodery stanowią fundamentalną klasę modeli głębokiego uczenia, zaprojektowaną w celu efektywnej ekstrakcji cech oraz uczenia reprezentacji danych w paradygmacie nienadzorowanym (ang. *unsupervised learning*) [14]. Architektura klasycznego autoenkodera opiera się na symetrii, składając się z dwóch głównych komponentów: **enkodera**, który mapuje dane wejściowe do przestrzeni o niższym wymiarze, oraz **dekodera**, którego zadaniem jest rekonstrukcja oryginalnych danych z tej skompresowanej postaci (Rysunek 2.1).



RYСУNEK 2.1: Ogólny schemat architektury autoenkodera. Dane wejściowe x są kompresowane do reprezentacji ukrytej (ang. *latent*) z , a następnie rekonstruowane jako x' . Warstwa latentna wymusza na modelu naukę najważniejszych cech danych.

Kluczowym pojęciem w tym kontekście jest **reprezentacja ukryta** (ang. *latent*). Jest to wektor lub tensor niskowymiarowy, generowany przez enkoder, który ma za zadanie zakodować najbardziej istotne atrybuty danych, eliminując jednocześnie szum i redundancję [2]. W idealnym scenariuszu, przestrzeń latentna charakteryzuje się dwiema pożądanymi cechami: **zwartością** oraz **ciągłością semantyczną**. Oznacza to, że małe przesunięcia w tej przestrzeni powinny odpowiadać płynnym zmianom w generowanych danych wyjściowych (np. stopniowa zmiana kąta nachylenia obiektu czy jego koloru), co umożliwia wykonywanie operacji takich jak interpolacja między próbkami.

W kontekście systemów neurosymbolicznych, rola przestrzeni latentnej ewoluuje. Przestaje być ona jedynie „czarną skrzynką” służącą kompresji, a staje się interfejsem pomiędzy surowymi danymi sensorycznymi (np. pikselami) a symbolicznymi parametrami modelu. Zamiast bezpośrednio rekonstruować piksele, dekodery (często nazywani w tym kontekście **modelerem** lub **rendererem**) interpretuje wektor latentny jako zestaw instrukcji lub parametrów geometrycznych (np. współrzędne środka, promień, kod RGB). Pozwala to na przejście od reprezentacji sub-symbolicznej do

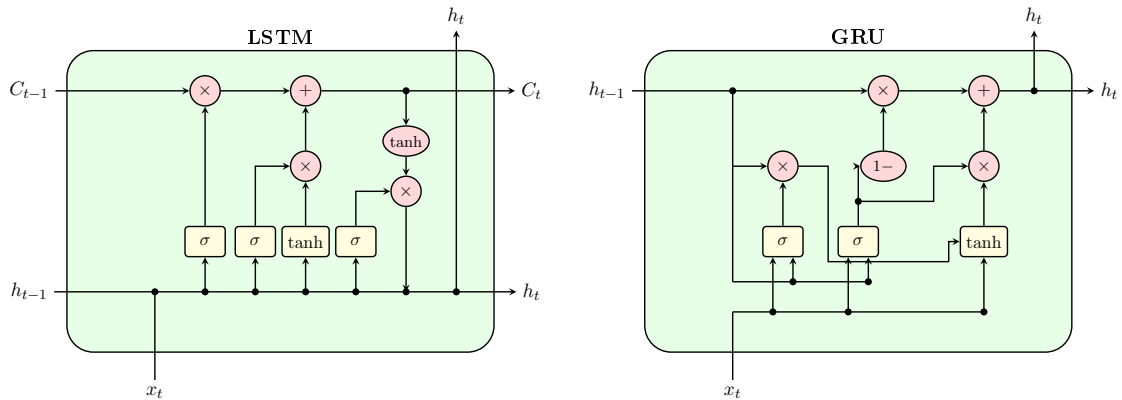
w pełni interpretowalnego opisu strukturalnego, co jest kluczowe dla zrozumiałości i edytowalności generowanych wyników.

2.2 Modelowanie sekwencyjne i sieci rekurencyjne

Podczas gdy splotowe sieci neuronowe (ang. *CNN*) stanowią standard w analizie danych o strukturze rastra (obrazy), modelowanie danych o charakterze sekwencyjnym wymaga odmiennych architektur. Wiele struktur występujących w obrazach rastrowych posiada naturalny charakter sekwencyjny. Dobrym przykładem są krawędzie oraz odcinki, które można interpretować jako uporządkowane sekwencje pikseli lub elementarnych podokien obrazu. Z tego względu problemy takie jak analiza rysunku odręcznego, śledzenie trajektorii ruchu czy obiektowa dekompozycja sceny wymagają podejścia uwzględniającego kolejność elementów.

Do przetwarzania tego typu danych wykorzystuje się **rekurencyjne sieci neuronowe** (ang. *RNN*). W odróżnieniu od sieci jednokierunkowych (ang. *feed-forward*), RNN posiadają wewnętrzny stan pamięci, który jest aktualizowany w każdym kroku czasowym. Pozwala to sieci na wykorzystanie informacji z przeszłości do podejmowania decyzji w teraźniejszości. Jednakże klasyczne sieci RNN obciążone są problemem zanikającego i eksplodującego gradientu, co w praktyce uniemożliwia im uczenie się zależności w długich sekwencjach [15].

Przełomem w tej dziedzinie stało się wprowadzenie architektury **Long Short-Term Memory** (ang. *LSTM*) przez Hochreitera i Schmidhubera [16]. LSTM rozwiązuje problem krótkotrwałej pamięci poprzez wprowadzenie komórki pamięci (ang. *cell state*) oraz mechanizmu bramkowania: bramki wejścia, wyjścia i zapominania (Rysunek 2.2). Bramki te decydują o tym, ile informacji z przeszłości zachować, a ile nowej informacji zapisać.



RYSUNEK 2.2: Schemat przepływu informacji oraz mechanizmów bramkowania w komórkach LSTM (po lewej) oraz GRU (po prawej). Opracowanie własne na podstawie [16, 17].

W systemach generatywnych operujących na obrazach, sieci rekurencyjne (zarówno LSTM, jak i ich uproszczona wersja GRU) często pełnią rolę **agenta decyzyjnego**. Taki agent iteracyjnie przemieszcza się po przestrzeni latentnej obrazu lub generuje sekwencję komend rysujących [8]. W każdej iteracji model podejmuje decyzję nie tylko o parametrach kolejnego elementu (np. „narysuj koło w punkcie X”), ale także o tym, czy zakończyć proces generowania. Pozwala to na elastyczne modelowanie struktur o zmiennej długości, co jest trudne do osiągnięcia przy użyciu standardowych architektur o stałym wyjściu.

Kluczowym elementem decydującym o efektywności sieci GRU (ang. *Gated Recurrent Unit*) w modelowaniu danych sekwencyjnych jest specyficzny układ nieliniowych równań różnicowych. Architektura ta wykorzystuje dwie bramki kontrolujące przepływ informacji, w odróżnieniu od trzech bramek stosowanych w klasycznym LSTM.

Niech x_t oznacza wektor wejściowy w kroku czasowym t , a h_{t-1} wektor stanu ukrytego z poprzedniego kroku. Mechanizm bramkowania definiują następujące równania:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (2.1)$$

gdzie z_t jest **bramką aktualizacji** (ang. *update gate*), określającą proporcję informacji, która zostanie przeniesiona z poprzedniego stanu w niezmienionej formie, a σ oznacza logistyczną funkcję aktywacji (Sigmoid), normalizującą wynik do przedziału $(0, 1)$. Zmienne W_z , U_z to macierze wag, a b_z to wektor obciążeń (bias).

Z kolei **bramka resetowania** (ang. *reset gate*) r_t , określająca, jak wiele informacji z przeszłości należy zignorować podczas obliczania kandydata na nowy stan ukryty, zdefiniowana jest jako:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (2.2)$$

Kandydat na nowy stan ukryty $\tilde{h}_t$, uwzględniający nowe wejście i odfiltrowaną przeszłość, wykorzystuje funkcję tangensa hiperbolicznego ($\tanh$):

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (2.3)$$

gdzie operator $\odot$ oznacza iloczyn Hadamarda (mnożenie wektorów element po elemencie).

Ostateczny stan ukryty h_t , przekazywany do kolejnego kroku czasowego $t + 1$ oraz stanowiący wyjście komórki, jest liniową kombinacją stanu poprzedniego oraz nowo wygenerowanego kandydata, ważoną przez bramkę aktualizacji:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (2.4)$$

Taka redukcja stopnia złożoności w stosunku do komórek LSTM sprawia, że sieć szybciej konverguje i jest mniej podatna na zjawisko przeuczenia na relatywnie małych, jednorodnych zbiorach geometrii.

2.3 Struktury kompozycyjne i problem wiązania

Reprezentacja świata jako zbioru niezależnych obiektów i relacji między nimi jest fundamentalna dla ludzkiego poznania. W kognitywistyce i sztucznej inteligencji koncepcja ta znana jest jako **kompozycyjność** (ang. *compositionality*) [7]. System kompozycyjny to taki, w którym znaczenie złożonej całości wynika bezpośrednio ze znaczenia jej części składowych oraz reguł ich łączenia.

W kontekście wizji komputerowej, kompozycyjność przejawia się w modelowaniu sceny nie jako „worka pikseli”, ale jako uporządkowanej struktury obiektów (np. drzewo składa się z pnia i korony, a korona z liści). Wyzwaniem pozostaje jednak tzw. **problem wiązania** (ang. *binding problem*) – w jaki sposób sieć neuronowa ma „zrozumieć”, że dany kolor (np. czerwony) i dany kształt (np. koło) należą do tego samego obiektu, a nie do obiektów sąsiednich [6].

Jednym z podejść do rozwiązania tego problemu jest wykorzystanie reprezentacji obiektocentrycznych (ang. *object-centric representations*), takich jak sekwencyjne generowanie prymitywów geometrycznych. Inspiracją dla tego typu modeli są m.in. **modele wypełniania przestrzeni** (ang. *space-filling models*), znane z chemii obliczeniowej, gdzie skomplikowane struktury molekularne reprezentowane są jako zbiory sfer (atomów) o określonych promieniach i pozycjach [18]. Adaptacja tej koncepcji do domeny 2D pozwala na reprezentowanie dowolnego kształtu jako skończonej sumy prostych figur geometrycznych.

2.4 Renderowanie różniczkowalne

W ogólnym ujęciu, renderowanie można zdefiniować jako proces transformacji wysokopoziomowego, symbolicznego opisu obiektu lub sceny (np. zbioru wektorów, poligonów i ich atrybutów) do jego widoku w dwuwymiarowej reprezentacji rastrowej (siatce pikseli).

Tradycyjne silniki renderujące (rasteryzatory) działają w sposób dyskretny, co czyni operację renderowania nieróżniczkowalną. Uniemożliwia to propagację gradientu i trenowanie modelu metodą spadku gradientu (ang. *backpropagation*). Rozwiązaniem tego problemu jest renderowanie różniczkowalne (ang. *differentiable rendering*), polegające na aproksymacji procesu rasteryzacji funkcjami ciągłymi [11].

Dzięki zastosowaniu rendererów różniczkowalnych możliwe staje się trenowanie systemów w pętli zamkniętej, gdzie sieć neuronowa uczy się generować parametry geometryczne bezpośrednio na podstawie błędu rekonstrukcji obrazu.

W kontekście modeli neurosymbolicznych, renderowanie różniczkowalne pełni rolę „dekodera deterministycznego”. Wymusza ono na sieci neuronowej generowanie parametrów, które są fizycznie poprawne i interpretowalne, ponieważ tylko takie pozwolą na wierną rekonstrukcję obrazu przez sztywny mechanizm renderera.

2.5 Mechanizmy uwagi

Chociaż rekurencyjne sieci neuronowe (LSTM/GRU) radzą sobie z sekwencjami lepiej niż proste sieci, ich wydajność spada wraz ze wzrostem długości sekwencji ze względu na konieczność kompresji całej historii do jednego wektora stanu. Rozwiązaniem tego ograniczenia są **mechanizmy uwagi** (ang. *attention mechanisms*), spopularyzowane pierwotnie w tłumaczeniu maszynowym [19].

W zadaniach wizualnych mechanizm uwagi pozwala modelowi dynamicznie „skupiać wzrok” na fragmentach danych wejściowych w każdym kroku czasowym. Zamiast przetwarzać cały obraz jednocześnie w identyczny sposób, model generuje mapę wag (ang. *attention map*), która określa istotność poszczególnych regionów obrazu dla aktualnie podejmowanej decyzji.

Jest to szczególnie użyteczne w modelach generujących sekwencje przestrzenne, ułatwiając lokalizację punktu startowego oraz zachowanie spójności lokalnej rysowanych struktur.

Formalnie, w klasycznym mechanizmie uwagi (np. uwaga Bahdanau), wektor kontekstu c_t dla danego kroku czasowego t obliczany jest jako ważona suma wektorów cech (reprezentujących fragmenty obrazu) h_i :

$$c_t = \sum_{i=1}^N \alpha_{t,i} h_i \quad (2.5)$$

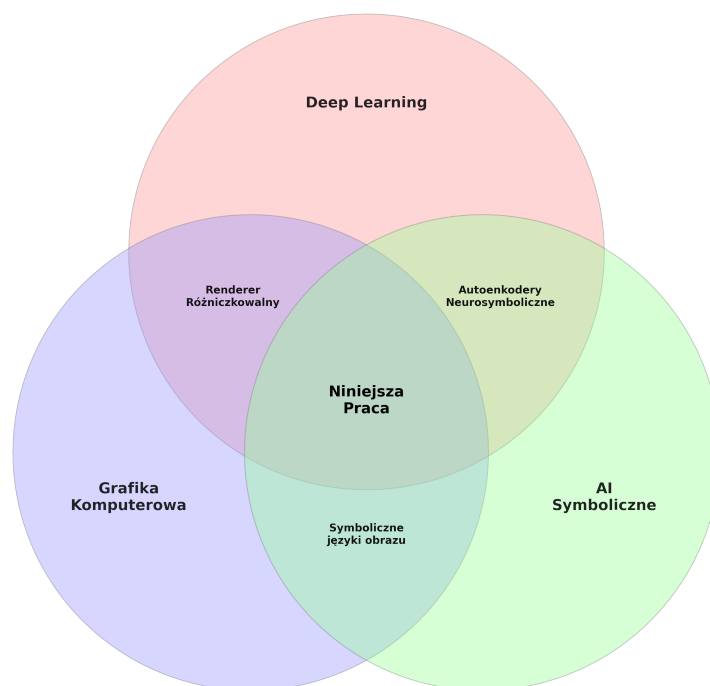
gdzie wagi uwagi $\alpha_{t,i}$ są generowane przez dodatkową podsieć, która ocenia dopasowanie (ang. *alignment*) pomiędzy bieżącym stanem ukrytym dekodera a danym fragmentem obrazu. Dzięki temu sieć uczy się ignorować szum tła i skupiać zasoby obliczeniowe wyłącznie na obszarach kluczowych dla aktualnie generowanego elementu.

Choć mechanizmy uwagi stanowią potężne narzędzie analityczne, w niniejszej pracy udowodniono, że dla określonej klasy struktur sekwencyjnych zastosowanie bezpośredniej dekompozycji siatkowej pozwala na uzyskanie równie wysokiej skuteczności przy znacznie mniejszej złożoności obliczeniowej.

Rozdział 3

Powiązane prace

W niniejszym rozdziale dokonano przeglądu literatury związanej z kluczowymi obszarami poruszonymi w pracy. Omówiono ewolucję metod uczenia reprezentacji, od klasycznych autoenkoderów po zaawansowane modele neurosymboliczne, ze szczególnym uwzględnieniem metod dekompozycji sceny, modelowania sekwencyjnego oraz renderowania różniczkowalnego.



RYSUNEK 3.1: Umieszczenie niniejszej pracy w kontekście powiązanych dziedzin badawczych.

3.1 Autoenkodery i uczenie reprezentacji obiektocentrycznych

Autoenkodery stanowią fundament nienadzorowanego uczenia reprezentacji. Ich klasyczna forma, wprowadzona przez Hintona i Salakhutdinova [14], służyła głównie do redukcji wymiarowości. Jed-

nak w kontekście analizy scen wizualnych, prosta kompresja całego obrazu do jednego wektora latentnego okazuje się niewystarczająca, gdyż gubi ona informację o strukturze poszczególnych obiektów. Przełomem w kierunku bardziej ustrukturyzowanych reprezentacji stały się **wariacyjne autoenkodery** (VAE) [20], które wprowadziły probabilistyczną naturę przestrzeni latentnej. Dalsze badania, takie jak β -VAE [21], pokazały, że poprzez odpowiednią regularyzację funkcji straty możliwe jest wymuszenie tzw. rozplątania (ang. *disentanglement*) cech, gdzie poszczególne wymiary wektora latentnego odpowiadają za izolowane cechy wizualne (np. osobno kolor, osobno rotacja). W ostatnich latach nurt ten wyewoluował w stronę **uczenia obiektocentrycznego** (ang. *object-centric learning*). Modele takie jak **MONet** (ang. *Multi-Object Network*) [22] czy **IODINE** [23] wykorzystują autoenkodery do iteracyjnej dekompozycji sceny na maski i obiekty, ucząc się wyodrębniać poszczególne elementy bez żadnego nadzoru etykietami. W niniejszej pracy, choć nie stosujemy pełnego podejścia probabilistycznego VAE, czerpiemy z idei dekompozycji sceny na niezależne, interpretowalne czynniki.

3.2 Modelowanie sekwencyjne i generowanie szkiców

Choć obrazy są strukturami statycznymi 2D, proces ich tworzenia (np. rysowanie) ma charakter sekwencyjny. Wykorzystanie rekurencyjnych sieci neuronowych (RNN) do modelowania tego procesu ma długą tradycję. Przełomowa praca Gravesa [9] pokazała, że sieci LSTM mogą skutecznie generować sekwencje pisma odręcznego, przewidując kolejne przesunięcia pióra (dx, dy) zamiast wartości pikseli. Najbardziej znaczącym punktem odniesienia dla niniejszej pracy jest model **SketchRNN** [8]. Ha i Eck zademonstrowali, że hybrydowa architektura VAE-LSTM potrafi skompresować cały rysunek wektorowy do jednego wektora latentnego, a następnie zrekonstruować go, generując sekwencję komend rysujących. SketchRNN udowodnił, że modelowanie obrazów jako sekwencji wektorów (ang. *stroke-based*) jest znacznie bardziej efektywne i semantycznie bogatsze niż podejście oparte na mapach pikseli.

W odróżnieniu od modelu SketchRNN, który opiera się na jednowymiarowych trajektoriach krawędziowych (zbliżonych koncepcyjnie do wektorów ścieżek SVG), proponowana w niniejszej pracy reprezentacja ma charakter wolumetryczny (powierzchniowy). Wykorzystanie sekwencji nakładających się na siebie, półprzezroczystych prymitywów dyskowych pozwala na znacznie lepszą adaptację do obiektów o niejasnych, zaszumionych lub rozmytych granicach. Reprezentacja oparta na krawędziach zawodzi, gdy obiekt pozbawiony jest wyraźnego konturu (co jest zjawiskiem powszechnym m.in. w obrazowaniu medycznym). Modelowanie poprzez iteracyjne wypełnianie obszaru masami pozwala na stabilniejszą aproksymację morfologii takich struktur.

3.3 Reprezentacje kompozycyjne i modele AIR

Fundamentalnym problemem w uczeniu maszynowym jest zrozumienie sceny jako kompozycji obiektów. Pierwszym modelem, który skutecznie połączył wnioskowanie wariacyjne z mechanizmem uwagi do dekompozycji sceny, był **Attend, Infer, Repeat** (AIR) [5]. Model ten wykorzystuje rekurencyjną sieć neuronową do sekwencyjnego "zliczania" i lokalizowania obiektów na obrazie, wycinając fragmenty obrazu (ang. *glimpses*) i przetwarzając je osobno. Koncepcję tę rozwinął Lin w modelu **SPACE** [24], który wyraźnie oddziela informację o tym "co" jest na obrazie (atrybuty obiektu), od tego "gdzie" to jest (pozycja). Podejście to (ang. *explicit spatial attention*) pozwala na budowanie modeli, które dobrze generalizują na sceny o innej liczbie obiektów niż w zbiorze treningowym. W kontekście prezentowanej pracy, inspiracją płynącą z modeli AIR i SPACE jest iteracyjny proces budowania sceny. Model, podobnie jak AIR, podejmuje w każdym

kroku decyzję o dodaniu kolejnego elementu, co pozwala na naturalną obsługę problemu zmiennej liczby obiektów.

3.4 Renderowanie różniczkowalne

Klasyczne graficzne potoki renderowania (rasteryzacja) są operacjami dyskretnymi, co uniemożliwia przepływ gradientu. Przełom nastąpił wraz z pojawieniem się technik **renderowania różniczkowalnego** (ang. *differentiable rendering*), które aproksymują proces rasteryzacji funkcjami ciągłymi. Kluczowym osiągnięciem jest tu biblioteka **DiffVG** autorstwa Li et al. [11], która umożliwia różniczkowanie względem parametrów krzywych wektorowych 2D. Dzięki temu możliwe stało się optymalizowanie kształtów wektorowych bezpośrednio na podstawie błędu rekonstrukcji pikseli, bez konieczności stosowania metod uczenia ze wzmocnieniem (ang. *reinforcement learning*).

Warto również podkreślić fundamentalną różnicę między popularnymi rendererami siatkowymi 3D, takimi jak Soft Rasterizer [25] czy Neural 3D Mesh Renderer [26], a zastosowanym w pracy podejściem 2D. Renderowanie obiektów trójwymiarowych narzuca konieczność wykonywania kosztownych obliczeniowo operacji rzutowania perspektywicznego, kalkulacji oświetlenia oraz rozwiązywania problemów zasłaniania (ang. *occlusion culling*). Ograniczenie przestrzeni roboczej wyłącznie do dwóch wymiarów pozwala na ominięcie rzutowania perspektywicznego, co drastycznie redukuje złożoność obliczeniową całego potoku uczenia. W niniejszej pracy zaprojektowano w pełni analityczny, lekki renderer 2D, który wykonuje operacje algebraiczne bezpośrednio na siatce pikseli, co umożliwia optymalizację *end-to-end* w czasie rzeczywistym.

3.5 Podejścia neurosymboliczne

Ostatnim, lecz kluczowym nurtem, są systemy **neurosymboliczne** (ang. *neurosymbolic AI*) [3, 4]. Łączą one percepcję neuronową (zdolność do radzenia sobie z zaszumionymi danymi wejściowymi) z rozumowaniem symbolicznym (logika, reguły, struktura). Przykładem takiego podejścia jest model **Neuro-Symbolic Concept Learner** (NS-CL) [27], który uczy się pojęć wizualnych (kolor, kształt) oraz gramatyki języka, łącząc sieć neuronową z silnikiem wykonywania programów symbolicznych. Innym przykładem są modele syntezy programów (ang. *program synthesis models*) [28], które traktują zadanie rozpoznawania obrazu jako problem syntezy programu komputerowego, który ten obraz wygeneruje. Niniejsza praca wpisuje się w ten nurt, proponując architekturę, w której sieć neuronowa pełni rolę "kompilatora", tłumaczącego surowy obraz na symboliczny "kod źródłowy" (listę parametrów geometrycznych), który jest następnie wykonywany przez deterministyczny renderer.

TABELA 3.1: Porównanie wybranych metod omawianych w literaturze z podejściem proponowanym w pracy.

Model / Metoda	Reprezentacja	Sekwencyjność	Nadzór	Renderowanie
Autoenkoder (klasyczny) [14]	Wektor (Latent)	Nie	Nie	Brak (Piksele)
SketchRNN [8]	Wektorowa (Strokes)	Tak (LSTM)	Tak (Szkice)	Nieróżniczkowalne
AIR [5]	Obiektowa (Glimpses)	Tak (RNN)	Nie	VAE (Piksele)
DiffVG [11]	Wektorowa (SVG)	Nie	-	Różniczkowalne
Proponowana Metoda	Symboliczna (Dyski)	Tak (LSTM/GRU)	Nie	Różniczkowalne

3.6 Przestrzenne reprezentacje kompozycyjne oparte na gęstych siatkach

Pomimo licznych zalet, modelowanie oparte na sieciach rekurencyjnych (RNN, GRU) narzuca sztuczny, sekwencyjny porządek na zbiory obiektów, które w rzeczywistości mogą nie posiadać jednoznacznej kolejności przestrzennej. Wymuszanie iteracyjnego wnioskowania staje się wąskim gardłem (ang. *bottleneck*) w sytuacjach, gdy struktury wizualne ulegają rozwidleniom (bifurkacjom) lub wielokrotnie się przecinają. Sieci rekurencyjne wykazują w takich scenariuszach tendencję do wpadania w nieskończone pętle lub przedwczesnego zatrzymywania predykcji.

Alternatywą rozwiązującą problem sekwencyjności są architektury operujące w dziedzinie czyisto przestrzennej, wykorzystujące mechanizm gęstych siatek (ang. *dense spatial grids*). Pionierskim rozwiązaniem, które zainspirowało ten nurt w widzeniu komputerowym, była architektura detekcyjna YOLO (ang. *You Only Look Once*) zaproponowana przez Redmona et al. [29]. W paradygmacie tym, zamiast generować obiekty krok po kroku lub stosować kosztowne algorytmy propozycji regionów (ang. *region proposal*), obraz wejściowy dzielony jest na regularną siatkę $S \times S$. Każda komórka tej siatki staje się w pełni niezależnym, równoległym predyktorem, odpowiedzialnym za wyestymowanie atrybutów obiektu (takich jak współrzędne środka, wymiary, czy prawdopodobieństwo wystąpienia) w swoim lokalnym polu recepcyjnym.

Adaptacja tego podejścia do generatywnych modeli neurosymbolicznych otwiera nowe możliwości w dekompozycji sceny. Przeniesienie ciężaru wnioskowania z osi czasu (kolejne kroki GRU) na oś przestrzenną (niezależne komórki macierzy wyjściowej) pozwala na całkowicie równoległe i zeterminowane generowanie zbioru prymitywów geometrycznych. Tego typu metody eliminują tzw. zagłodzenie przestrzeni latentnej (ang. *latent starvation*), w którym początkowy błąd w sekwencji propaguje się na jej koniec.

W niniejszej pracy koncepcję tę rozwinęto poprzez opracowanie przestrzennego dekodera siatkowego połączonego z różniczkowalnym renderowaniem. Pozwala to modelowi na swobodne, nie-nadzorowane budowanie rozgałęzionych struktur z nałożonych na siebie, półprzezroczystych elementów przestrzennych, co skutecznie przełamuje ograniczenia natury topologicznej napotykaną w klasycznym wnioskowaniu rekurencyjnym.

Rozdział 4

Proponowana metoda

W niniejszym rozdziale przedstawiono szczegółowy projekt zrealizowanego systemu neurosymbolicznego. Omówiono proces przygotowania danych, architekturę sieci neuronowej oraz zdefiniowano funkcję celu sterującą procesem uczenia.

4.1 Generowanie syntetycznego zbioru danych

Kluczowym elementem metodyki jest opracowanie parametrycznego generatora danych syntetycznych. W przeciwieństwie do standardowych zbiorów danych wizualnych, takich jak MNIST czy CIFAR [30, 31], zastosowanie generatora pozwala na pełną kontrolę nad złożonością sceny oraz zapewnia dostęp do precyzyjnych danych referencyjnych (ang. *ground truth*) w postaci parametrów geometrycznych.

Zaprojektowano środowisko symulacyjne `SnakeGeometricDataset`, które generuje obrazy o rozdzielczości 128×128 pikseli, przedstawiające sekwencyjne struktury geometryczne przypominające łańcuchy lub „węże”. Struktury te składają się z prymitywów (kół lub elips) ułożonych w ciągłą ścieżkę.

4.1.1 Algorytm generacji sekwencyjnej

Proces tworzenia pojedynczej próbki danych opiera się na zmodyfikowanym procesie błądzenia losowego z ograniczeniami fizycznymi (ang. *constrained random walk*). Algorytm przebiega w następujących krokach:

1. **Inicjalizacja:** Losowana jest długość sekwencji N z przedziału $[N_{min}, N_{max}]$, punkt startowy (x_0, y_0) oraz początkowy kąt ruchu α_0 . System umożliwia wymuszenie startu w centrum obrazu lub w losowej pozycji z zachowaniem marginesu bezpieczeństwa.
2. **Iteracyjny przyrost:** Pozycja każdego kolejnego elementu i zależy od pozycji elementu poprzedniego $i - 1$, zadanego promienia obiektów oraz kierunku ruchu. Aby zapewnić naturalny, wijący się kształt struktury, kąt ruchu α_i jest modyfikowany w każdym kroku o losową wartość $\Delta\alpha$ (momentum):

$$\alpha_i = \alpha_{i-1} + \text{uniform}(-\delta_{turn}, \delta_{turn}) \quad (4.1)$$

gdzie δ_{turn} to parametr sterujący krętością węża (ang. *turn severity*).

3. **Fizyka odbić i granic:** Zaimplementowano mechanizm wykrywania kolizji z krawędziami obrazu. W przypadku, gdy koleny element miałby wyjść poza kadr, algorytm symuluje fi-

yczne odbicie wektora ruchu (zmieniając odpowiednio składową kąta), co pozwala na generowanie długich struktur zamkniętych w ograniczonej przestrzeni bez przerywania ciągłości.

4. **Próbkowanie z odrzucaniem (ang. *rejection sampling*):** Aby zagwarantować poprawność topologiczną (np. brak niepożądanych samoprzecięć w trybie *non-intersecting*), zastosowano strategię próbkowania z odrzucaniem. Jeśli nowo wygenerowany element narusza warunki (np. koliduje z ogonem węża), algorytm podejmuje do 30 prób wylosowania innego kąta. W przypadku niepowodzenia (tzw. utknięcie w ślepym zaułku), cała sekwencja jest odrzucana, a proces generowania rozpoczyna się od nowa. Gwarantuje to, że do modelu trafiają wyłącznie poprawne i kompletne próbki.

Działanie zmodyfikowanego procesu błędzenia losowego, wymuszającego zachowanie poprawności geometrycznej, można zapisać w postaci sformalizowanej. Algorytm 1 przedstawia krok po kroku logikę decyzyjną pętli głównej generatora uwzględniającą proces próbkowania z odrzucaniem (ang. *Rejection Sampling*).

Algorithm 1 Algorytm generacji z użyciem Rejection Sampling

Require: Długość sekwencji N , margines granic m , max. prób K

Ensure: Zbiór poprawnych parametrów $\mathbf{Y}$ lub błąd

```

1: ZAINICJUJ  $\mathbf{p}_0 \leftarrow (0.5, 0.5)$ 
2:  $\alpha_0 \leftarrow U(0, 2\pi)$ 
3:  $\mathbf{Y} \leftarrow [(\mathbf{p}_0, \alpha_0, r_{major}, r_{minor}, color)]$ 
4: for  $i = 1$  to  $N - 1$  do
5:    $success \leftarrow \text{FALSE}$ 
6:   for  $k = 1$  to  $K$  do
7:      $\Delta\alpha \leftarrow U(-\delta_{turn}, \delta_{turn})$ 
8:      $\alpha_i \leftarrow \alpha_{i-1} + \Delta\alpha$ 
9:      $\mathbf{p}_i \leftarrow \mathbf{p}_{i-1} + step \cdot [\cos(\alpha_i), \sin(\alpha_i)]$ 
10:    if  $\mathbf{p}_i$  poza granicami  $[m, 1 - m]$  then
11:      Symuluj odbicie:  $\alpha_i \leftarrow \text{ModifyForReflection}(\alpha_i, \mathbf{p}_i)$ 
12:      Przelicz  $\mathbf{p}_i$  na nowo
13:    end if
14:    if not  $\text{CheckCollision}(\mathbf{p}_i, \mathbf{Y}, r_{major})$  then
15:      Dodaj  $\mathbf{p}_i, \alpha_i$  do  $\mathbf{Y}$ 
16:       $success \leftarrow \text{TRUE}$ 
17:      break
18:    end if
19:  end for
20:  if not  $success$  then
21:    return BŁĄD_ŚLEPY_ZAUŁEK {Odrzuć całą sekwencję}
22:  end if
23: end for
24: return  $\mathbf{Y}$ 

```

4.1.2 Parametryzacja geometrii i wyglądu

Generator umożliwia sterowanie poziomem trudności zadania poprzez szereg hiperparametrów. Umożliwia to zastosowanie strategii uczenia programowanego (ang. *curriculum learning*), polegającej na stopniowym zwiększaniu trudności danych w trakcie treningu.

- **Orientacja elips (ang. *tangent flow*):** W przypadku generowania elips, zaimplementowano mechanizm styczności. Kąt rotacji elipsy θ nie jest losowy, lecz jest ściśle powiązany z wektorem prędkości węża w danym punkcie. Dzięki temu elipsy układają się wzdłuż ścieżki, tworząc spójne, organiczne kształty, co stanowi dodatkową wskazówkę geometryczną dla sieci.
- **Tryby kolorystyczne:**
 - *Random*: Każdy obiekt ma niezależny, losowy kolor RGB.
 - *Constant*: Cała struktura posiada jedną, losową barwę.
 - *Fade (gradient)*: Kolor C_i kolejnego elementu jest wynikiem niewielkiej perturbacji koloru C_{i-1} . Modeluje to płynne przejścia tonalne, co uczy sieć wnioskowania o kolejności elementów w sekwencji.

4.1.3 Reprezentacja danych wejściowych i wyjściowych

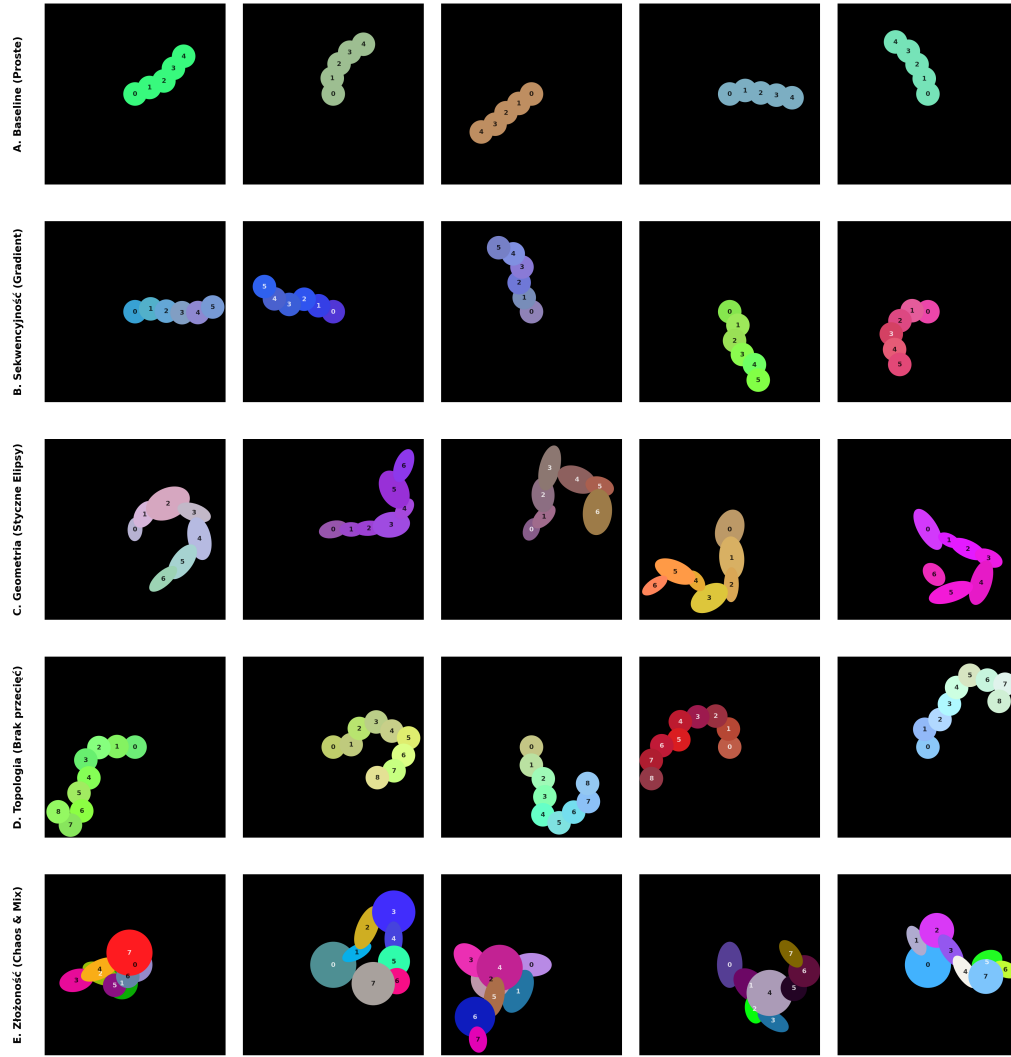
Proces generacji danych tworzy pary (Y, X) , gdzie przepływ informacji następuje od reprezentacji symbolicznej do rastrowej:

- $Y \in \mathbb{R}^{N_{max} \times 9}$ – tensor parametrów opisujący scenę (ang. *ground truth*), generowany stochastycznie przez opisywany algorytm.
- $X \in \mathbb{R}^{128 \times 128 \times 3}$ – obraz rastrowy RGB, powstały poprzez wyrenderowanie parametrów Y .

Każdy wiersz tensora Y opisuje jeden prymityw geometryczny za pomocą 9 znormalizowanych wartości:

$$y_i = [c_x, c_y, r_{major}, r_{minor}, \theta, R, G, B, EOS] \quad (4.2)$$

gdzie (c_x, c_y) to współrzędne środka, r_{major} i r_{minor} to promienie elipsy, θ to kąt rotacji, a R, G, B to składowe koloru. Flaga EOS (ang. *end of sequence*) przyjmuje wartość 1.0 dla elementów wypełniających (ang. *padding*) oraz ostatniego elementu sekwencji, sygnalizując koniec struktury. Warto zaznaczyć, że w trakcie treningu modelu neurosymbolicznego kierunek przetwarzania jest odwrotny (od X do Y).



RYSUNEK 4.1: Wizualizacja możliwości parametrycznego generatora danych syntetycznych. Obraz prezentuje pięć poziomów złożoności danych (wiersze A-E), demonstrując kontrolę nad cechami ukrytymi (ang. *latent features*). **A:** Wariant podstawowy (ang. *baseline*) – stały rozmiar i kolor, niska złożoność. **B:** Sekwencyjność – wykorzystanie gradientu kolorystycznego (ang. *fade*) do wizualnego zakodowania kierunku i kolejności elementów. **C:** Geometria – orientacja elips jest dynamicznie dopasowywana do wektora prędkości (styczność), co wymusza spójność kształtu. **D:** Topologia – algorytm próbkowania z odrzucaniem wymusza generowanie długich struktur bez samoprzecięć. **E:** Złożoność (ang. *chaos*) – wariant łączący zmienne kształty, losowe kolory oraz chaotyczną dynamikę ruchu z dozwolonymi pętlami, służący do testowania generalizacji modelu.

4.1.4 Rozszerzenie generatora o topologie złożone i skrzyżowania

Standardowy algorytm błędzenia losowego z próbkowaniem z odrzucaniem (opisany w podrozdziale 4.1.1) został zoptymalizowany pod kątem generowania pojedynczych, nieprzecinających się łańcuchów. Jednakże, aby rzetelnie ewaluować zdolności modeli neurosymbolicznych w kontekście docelowych zastosowań medycznych (takich jak analiza sieci naczyń krwionośnych), konieczne było rozszerzenie generatora o obsługę struktur grafowych o wysokim stopniu komplikacji.

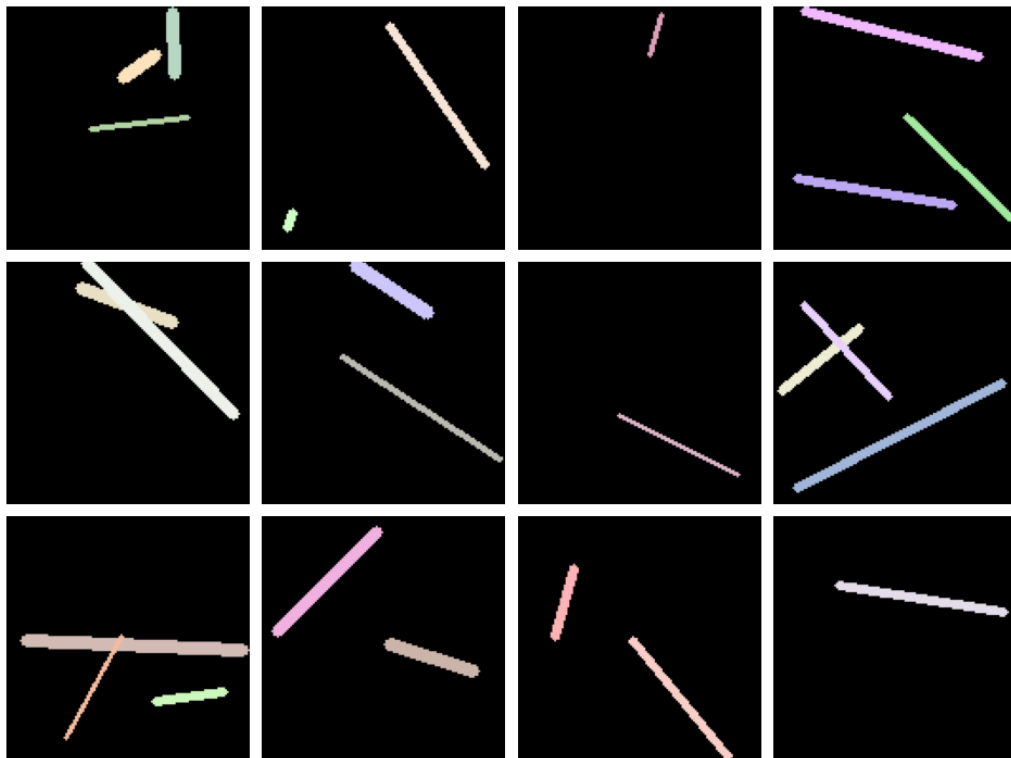
W tym celu zaimplementowano zaawansowany wariant generatora, pozwalający na tworzenie tzw. topologii złożonych. Rozszerzenie to wprowadza dwa nowe paradygmaty generacji obrazu:

- **Niezależne trajektorie przecinające się (Skrzyżowania typu X):** Zamiast jednej sekwencji, algorytm inicjuje K niezależnych ścieżek o losowych punktach startowych, wyłąc-

czając jednocześnie mechanizm unikania kolizji (ang. *collision avoidance*). Pozwala to na naturalne krzyżowanie się obiektów, generując struktury, w których prymitywy wizualnie nakładają się na siebie.

- **Rozwidlenia trajektorii (Bifurkacje typu T i Y):** Algorytm posiada zdefiniowane prawdopodobieństwo podziału aktywnej ścieżki. W punkcie węzłowym (ang. *branching point*) wektor ruchu ulega rozszczepieniu, a generator rozpoczyna symultaniczne rysowanie dwóch nowych gałęzi pod różnymi kątami.

Wygenerowany w ten sposób zbiór danych (w środowisku programistycznym określany mianem wariantu o podwyższonym stopniu trudności) charakteryzuje się brakiem jednoznacznej kolejności elementów, co celowo łamie założenia klasycznych struktur sekwencyjnych. Wzbogacenie tych obrazów o ekstremalny szum sensorowy stanowiło ostateczne i najbardziej rygorystyczne środowisko testowe dla projektowanych w pracy architekt dekodów.

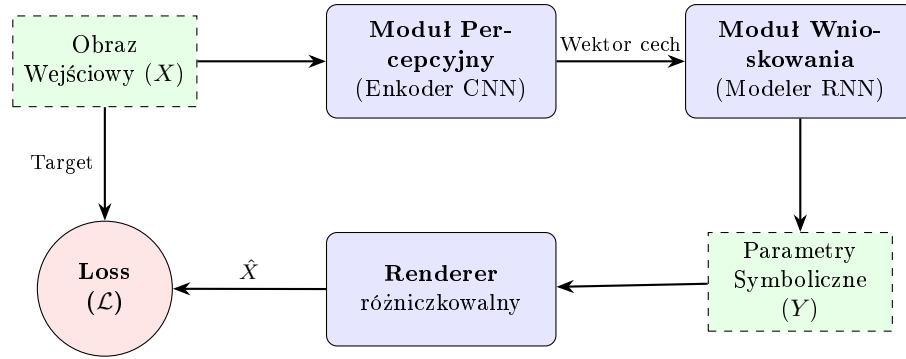


RYSUNEK 4.2: Przykładowe instancje ze zbioru danych o podwyższonym stopniu trudności (maski referencyjne). Widoczne wygenerowane losowo niezależne trajektorie przecinające się (skrzyżowania typu X) oraz rozwidlenia (bifurkacje typu T), łamiące założenia o jednoznacznej sekwencyjności obiektów.

4.2 Ogólna architektura systemu

Zaproponowane rozwiązanie opiera się na paradygmacie uczenia neurosymbolicznego, łączącym elastyczność głębokich sieci neuronowych z interpretowalnością reprezentacji symbolicznych. System realizuje zadanie tzw. odwrotnej grafiki komputerowej (ang. *inverse graphics*), gdzie celem jest odtworzenie parametrów generatywnych sceny na podstawie jej obrazu rastrowego.

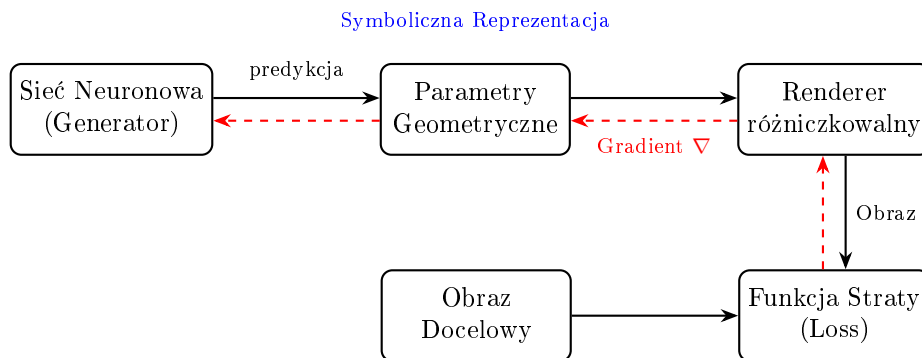
Architektura systemu składa się z trzech głównych, różniczkowalnych modułów, tworzących potok przetwarzania typu *end-to-end*. Schemat przepływu danych przedstawiono na Rysunku 4.3.



RYSUNEK 4.3: Schemat blokowy proponowanej architektury neurosymbolicznej. System przetwarza obraz wejściowy na sekwencję parametrów, a następnie weryfikuje poprawność predykcji poprzez rekonstrukcję obrazu i porównanie go z oryginałem w module funkcji kosztu.

Proces przetwarzania przebiega w następujących etapach:

1. **Percepcja (Enkoder):** Obraz wejściowy X jest przetwarzany przez splotową sieć neuronową (CNN). Zadaniem tego modułu jest ekstrakcja niskopoziomowych cech wizualnych (krawędzie, kolory) oraz zbudowanie abstrakcyjnej, wektorowej reprezentacji całej sceny.
2. **Wnioskowanie sekwencyjne (Dekoder):** Wektor cech trafia do sieci rekurencyjnej (RNN/LSTM). Moduł ten działa w pętli, generując krok po kroku parametry y_t dla kolejnych elementów struktury (kół lub elips). Sieć rekurencyjna odpowiada za zrozumienie zależności przestrzennych, takich jak ciągłość węża czy następstwo elementów.
3. **Rendering i optymalizacja:** Wygenerowane parametry symboliczne Y są przekazywane do renderera. W przeciwieństwie do klasycznych silników graficznych, zastosowany renderer jest w pełni różniczkowalny. Pozwala to na zrekonstruowanie obrazu $\hat{X}$ i obliczenie funkcji straty $\mathcal{L}(X, \hat{X})$. Dzięki różniczkowalności, gradient błędu może zostać propagowany wstecznie (ang. *backpropagation*) przez renderer, aż do wag sieci neuronowej, umożliwiając uczenie całego systemu.



RYSUNEK 4.4: Schemat uczenia z wykorzystaniem renderera różniczkowalnego. Czerwona przerywana linia obrazuje przepływ gradientu, który zgodnie z propagacją wsteczną płynie od funkcji straty, przez renderer i parametry geometryczne, z powrotem do sieci neuronowej.

Taka architektura pozwala modelowi uczyć się nie tylko rozpoznawania obiektów, ale także rozumienia ich struktury i wzajemnych relacji, dążąc do minimalizacji różnicy między obrazem rzeczywistym a jego cyfrową rekonstrukcją.

4.3 Moduł percepcyjny (Enkoder)

Rolę układu wizyjnego w systemie pełni moduł percepcyjny, zrealizowany jako głęboka spłotowa sieć neuronowa (CNN). Jego zadaniem jest kompresja wysokowymiarowego obrazu wejściowego $X \in \mathbb{R}^{128 \times 128 \times 3}$ do zwartego wektora cech ukrytych (ang. *latent context vector*) $C \in \mathbb{R}^{128}$. Wektor ten stanowi abstrakcyjny, numeryczny opis całej sceny, który jest następnie wykorzystywany przez dekodery do inicjalizacji procesu generowania sekwencji.

Zdecydowano się na zaprojektowanie dedykowanej, lekkiej architektury zamiast wykorzystania gotowych modeli (np. ResNet), ze względu na geometryczny charakter danych i potrzebę zachowania efektywności obliczeniowej.

4.3.1 Architektura sieci

Enkoder składa się z czterech bloków spłotowych. W każdym bloku zastosowano spłot z filtrem o rozmiarze 4×4 , paddingiem 1 oraz krokiem spłotu (ang. *stride*) równym 2. Zastosowanie spłotów ze skokiem (ang. *strided convolutions*) zamiast klasycznych warstw łączących (np. *max pooling*) pozwala sieci na adaptacyjne uczenie się operacji redukcji wymiarowości (ang. *downsampling*), co jest korzystne przy zachowywaniu informacji o położeniu przestrzennym obiektów. Decyzja o zastosowaniu parzystych filtrów o rozmiarze 4×4 z krokiem 2 wynika z faktu, że umożliwiają one bezresztowe i symetryczne dzielenie wymiarów obrazu przez potęgę dwójki w kolejnych warstwach.

Po przejściu przez warstwy spłotowe, mapa cech jest spłaszczana (ang. *flatten*) i przetwarzana przez dwie warstwy w pełni połączone (MLP). Szczegółową konfigurację warstw przedstawiono w Tabeli 4.1.

TABELA 4.1: Szczegółowa konfiguracja warstw modułu percepcyjnego (enkodera). Funkcja aktywacji ReLU jest stosowana po każdej warstwie z wyjątkiem ostatniej.

Typ warstwy	Parametry (Kernel / Stride)	Rozmiar wyjściowy	Kanały
Wejście	-	128×128	3 (RGB)
Conv2d	$4 \times 4 / 2$	64×64	32
Conv2d	$4 \times 4 / 2$	32×32	64
Conv2d	$4 \times 4 / 2$	16×16	128
Conv2d	$4 \times 4 / 2$	8×8	256
Flatten	-	16384	-
Dense (Ukryta)	-	512	-
Dense (Wyjście)	-	128	-

Zastosowana architektura spłotowa charakteryzuje się precyzyjnie dobranym globalnym polem recepcyjnym (ang. *receptive field*). Zdolność enkodera do „widzenia” całego obrazu w jednym przebiegu determinowana jest przez parametry warstw (kernel $K = 4$, stride $S = 2$). Pole recepcyjne po n -tej warstwie spłotowej można obliczyć z zależności rekurencyjnej:

$$RF_n = RF_{n-1} + (K_n - 1) \prod_{i=1}^{n-1} S_i \quad (4.3)$$

gdzie $RF_0 = 1$. Obliczenia dla zastosowanej kaskady czterech warstw spłotowych wskazują, że pojedynczy neuron na wyjściu ostatniej warstwy spłotowej przed operacją spłaszczania (Flatten) obejmuje swoim polem recepcyjnym niemal całą powierzchnię pierwotnego obrazu 128×128 . To matematyczne podłoże tłumaczy skuteczność z jaką model buduje holistyczny kontekst C , niezbędny dla sieci rekurencyjnej do zainicjowania procesu wnioskowania o współrzędnych bez jawnego skanowania obrazu przy użyciu okna przesuwającego.

4.3.2 Przestrzeń ukryta

Jako funkcję aktywacji w warstwach ukrytych zastosowano ReLU, co zapewnia nieliniowość i przyspiesza zbieżność treningu. Kluczowym elementem jest jednak zastosowanie funkcji tangens hiperboliczny (**Tanh**) na wyjściu ostatniej warstwy liniowej:

$$C = \tanh(W_2 \cdot \text{ReLU}(W_1 \cdot x_{flat} + b_1) + b_2) \quad (4.4)$$

gdzie C to wektor kontekstu wyjściowego, W_1 i W_2 to macierze wag kolejno pierwszej i drugiej warstwy liniowej (dense), x_{flat} to wektor cech po operacji spłaszczania, a b_1 i b_2 to odpowiadające wektory obciążeń (ang. *bias*).

Zastosowanie Tanh normalizuje wartości wektora kontekstu do przedziału $[-1,1]$. Jest to zabieg celowy, mający na celu stabilizację procesu uczenia modułu rekurencyjnego (RNN), dla którego znormalizowane wejścia są łatwiejsze do interpretacji niż nieograniczone wartości generowane przez funkcję liniową. Wektor C zawiera skompresowaną informację o globalnych cechach obrazu wejściowego. Wybór wymiarowości wektora ukrytego $C = 128$ podyktowany był koniecznością stworzenia odpowiedniego wąskiego gardła (ang. *bottleneck*) reprezentacji. Przy założonej maksymalnej długości sekwencji (10 elementów) i 9 parametrach na obiekt, łączna liczba stopni swobody wynosi 90. Wymiar 128 zapewnia wystarczającą pojemność informacyjną do zakodowania tych atrybutów z marginesem na redundancję, jednocześnie wymuszając na modelu ekstrakcję tylko najbardziej istotnych cech wizualnych. Eksperymenty wstępne wykazały, że niższe wymiarowości (np. 64) prowadziły do utraty detali kolorystycznych, natomiast wyższe (256) zwiększały ryzyko przeuczenia.

4.4 Moduł wnioskowania (Modeler)

Sercem systemu generatywnego jest moduł wnioskowania (ang. *modeler* lub *symbolic reasoner*), zrealizowany w oparciu o rekurencyjną sieć neuronową (RNN). Jego zadaniem jest przetłumaczenie statycznego wektora kontekstu C (wygenerowanego przez enkoder) na uporządkowaną sekwencję parametrów geometrycznych Y .

Moduł wnioskowania działa jak agent, który "wyobraża sobie" proces rysowania węża krok po kroku. W ramach prac badawczych przeanalizowano przydatność dwóch wiodących architektur dla tego modułu: LSTM [16] oraz GRU [17].

4.4.1 Wybór architektury rekurencyjnej (GRU vs LSTM)

Obie rozważane struktury (LSTM i GRU) zostały zaprojektowane w celu rozwiązania problemu znikających gradientów, co pozwala im na efektywne modelowanie długoterminowych zależności w danych sekwencyjnych.

Przeprowadzone eksperymenty wstępne wykazały jednak wyraźną przewagę jednostek GRU w kontekście analizowanego problemu geometrycznego. Zaobserwowano następujące korzyści wynikające z zastosowania GRU:

1. **Efektywność parametrów:** Komórka GRU posiada uproszczoną strukturę (dwie bramki sterujące zamiast trzech w LSTM), co przekłada się na mniejszą liczbę wag do wytrenowania. Przy ograniczonej wielkości zbioru danych, pozwala to na szybszą zbieżność modelu i zmniejsza ryzyko przeuczenia.
2. **Stabilność generacji:** W toku badań empirycznych model oparty na GRU wykazywał mniejszą tendencję do generowania niestabilnych trajektorii (tzw. oscylacji węża) w porównaniu do sieci LSTM o zbliżonej pojemności informacyjnej.

Z tego względu w finalnej architekturze zdecydowano się na zastosowanie dwuwarstwowego układu GRU.

4.4.2 Mechanizm integracji kontekstu

Aby wzmocnić spójność generowanych struktur, zastosowano mechanizm ciągłej integracji kontekstu. W każdym kroku czasowym t , wejście do komórki rekurencyjnej nie jest surowym parametrem z poprzedniego kroku y_{t-1} , lecz wynikiem przetworzenia przez warstwę integracji:

$$x_t^{rnn} = \text{LayerNorm}(\text{ReLU}(W_f \cdot [y_{t-1}, C] + b_f)) \quad (4.5)$$

gdzie x_t^{rnn} to zintegrowane wejście dla kroku t , $[\dots]$ oznacza konkatencję wektorów, W_f i b_f to wagi i obciążenia warstwy fuzji (ang. *fusion layer*). Zmienna C to kontekst rozumiany jako globalna, ukryta reprezentacja całego obrazu wejściowego, wyekstrahowana wcześniej przez enkoder CNN, a y_{t-1} to parametry wygenerowane w poprzednim kroku czasowym. Dzięki temu sieć w każdym kroku decyzyjnym ma stały dostęp do globalnego obrazu sytuacji, co zapobiega „zapominaniu” celu generacji w miarę wydłużania się sekwencji.

4.4.3 Dedykowane głowice wyjściowe

Wyjście z warstwy rekurencyjnej jest przetwarzane przez dwie niezależne głowice (podsieci MLP), co pozwala na rozdzielenie odpowiedzialności za różne aspekty generacji:

- **Głowica ruchu (ang. *head move*):** Odpowiada za przewidywanie relatywnego przesunięcia $(\Delta x, \Delta y)$. Zastosowano tu funkcję aktywacji **Tanh** skalowaną współczynnikiem 0.5. Ogranicza to maksymalny krok węża w jednej iteracji, co stabilizuje proces uczenia.
- **Głowica parametrów (ang. *head params*):** Odpowiada za przewidywanie promieni (r_{major}, r_{minor}) , kąta rotacji (θ) oraz składowych koloru (R, G, B) . Zastosowano tu funkcję **Sigmoid**, aby zagwarantować, że wygenerowane wartości zawsze mieszczą się w poprawnym zakresie $[0, 1]$.

4.5 Reprezentacja wyjścia i dynamika ruchu

Istotnym aspektem zaproponowanej metody jest sposób, w jaki dekodery parametryzuje trajektorię obiektu. Analiza kodu źródłowego i eksperymenty wykazały, że bezpośrednia predykcja współrzędnych absolutnych (x, y) jest trudna dla sieci rekurencyjnych i prowadzi do niestabilności.

Dlatego w niniejszej pracy zastosowano modelowanie przyrostowe (ang. *relative movement modeling*). Dekoder w każdym kroku t generuje wektor przesunięcia $\Delta \mathbf{p}_t$, a ostateczna pozycja $\mathbf{p}_t$ jest sumą kumulacyjną:

$$\mathbf{p}_t = \mathbf{p}_{t-1} + \Delta \mathbf{p}_t, \quad \text{gdzie} \quad \Delta \mathbf{p}_t = [\Delta x_t, \Delta y_t] \quad (4.6)$$

Pozostałe atrybuty (promień r_t , kolor c_t oraz sygnał końca sekwencji eos_t) są przewidywane w sposób bezpośredni. Taka dekompozycja problemu naśladuje fizykę ruchu, gdzie obiekt przemieszcza się z określoną prędkością i kierunkiem, zachowując (lub płynnie zmieniając) swoje cechy wizualne.

4.6 Kryteria agregowane funkcji straty z maskowaniem sekwencji

Podczas przetwarzania struktur o zmiennej długości w układach wsadowych (ang. *batches*), konieczne jest wyrównanie wymiarów tensorów. W tym celu stosuje się tzw. wypełniacz (ang. *pad-*

ding) – sztuczne elementy dodawane na końcu krótszych sekwencji, aby zrównać ich długość z maksymalną sekwencją w paczce.

Aby skutecznie trenować model, nie karząc go za przewidywania w miejscach wypełniaczy, zdefiniowano złożoną funkcję straty (ang. *loss function*), która wykorzystuje mechanizm maskowania aktywności. Maskę M_t przyjmuje wartość 1 dla rzeczywistych elementów sekwencji i 0 dla elementów typu *padding*. Całkowita strata $\mathcal{L}_{total}$ jest ważoną sumą pięciu składników:

$$\mathcal{L}_{total} = \lambda_{pos}\mathcal{L}_{pos} + \lambda_{phys}\mathcal{L}_{phys} + \lambda_{size}\mathcal{L}_{size} + \lambda_{col}\mathcal{L}_{col} + \lambda_{stop}\mathcal{L}_{stop} \quad (4.7)$$

W poniższych definicjach wielkości z daszkiem (np. $\hat{p}_t$) oznaczają predykcje wygenerowane przez model, natomiast zmienne bez daszka (np. p_t) stanowią wartości docelowe (ang. *target*). Współczynniki λ sterują proporcjonalnym wpływem poszczególnych błędów na końcowy gradient.

Poszczególne komponenty zdefiniowano następująco:

1. **Błąd trajektorii ($\mathcal{L}_{pos}$):** Maskowany błąd średniokwadratowy (MSE) relatywnego przesunięcia:

$$\mathcal{L}_{pos} = \frac{1}{N} \sum_t M_t \cdot \|\Delta\hat{p}_t - \Delta p_t\|^2 \quad (4.8)$$

Waga $\lambda_{pos} = 80.0$ nadaje priorytet poprawności geometrycznej.

2. **Błąd nieciągłości ($\mathcal{L}_{phys}$):** Rygorystyczne ograniczenie zapobiegające zjawisku "rozrywania" węzła. Kara nakładana jest na sytuacje, w których wygenerowane przesunięcie $\Delta\hat{p}_t$ nadmiernie oddala elementy od siebie względem referencyjnego dystansu Δp_t :

$$\mathcal{L}_{phys} = \frac{1}{N} \sum_t M_t \cdot ||\Delta\hat{p}_t||_2 - ||\Delta p_t||_2| \quad (4.9)$$

3. **Błąd rozmiaru ($\mathcal{L}_{size}$) i koloru ($\mathcal{L}_{col}$):** Obliczane jako maskowany błąd średniokwadratowy (MSE) dla wartości wektora promieni oraz błąd absolutny (L1) dla wektora barw *RGB*. Wykorzystanie błędu L1 dla koloru pozwala zachować wyższą wierność przejść tonalnych.
4. **Błąd stopu ($\mathcal{L}_{stop}$):** Krytyczny element decydujący o długości sekwencji. Różnica między przewidzianym prawdopodobieństwem końca $e\hat{o}s_t$ a referencyjną flagą eos_t liczona jest za pomocą entropii krzyżowej (ang. *binary cross-entropy*, BCE). Silnie karze to model zarówno za zbyt wczesne zakończenie generowania, jak i za kontynuowanie "rysowania" po wyczerpaniu docelowej struktury ($\lambda_{stop} = 80.0$).

4.7 Matematyczny model renderowania różniczkowalnego

Kluczowym komponentem umożliwiającym przepływ gradientu (ang. *gradient flow*) w potoku end-to-end jest autorski moduł renderera. Klasyczna rasteryzacja, polegająca na dyskretnym przypisywaniu wartości pikselom (np. algorytm Bresenhama), jest operacją nieróżniczkowalną. Aby ominąć tę barierę, proces rysowania prymitywów geometrycznych (elips) został zamodelowany w dziedzinie ciągłej przy użyciu aproksymacji gładkich.

Niech (x, y) oznaczają współrzędne znormalizowanej siatki pikseli obrazu, gdzie $x, y \in [0, 1]$. Pojedyncza elipsa w przestrzeni zdefiniowana jest przez pięć parametrów przewidywanych przez sieć: środek (c_x, c_y) , promienie półosi (r_{major}, r_{minor}) oraz kąt rotacji θ . W pierwszym kroku układ współrzędnych siatki jest przesuwany i obracany względem środka lokalnego elipsy:

$$\begin{bmatrix} x_{trans} \\ y_{trans} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x - c_x \\ y - c_y \end{bmatrix} \quad (4.10)$$

Dystans algebraiczny $d(x, y)$ danego piksela od środka elipsy wyrażony jest znormalizowanym równaniem kwadratowym:

$$d(x, y) = \left(\frac{x_{trans}}{r_{major}} \right)^2 + \left(\frac{y_{trans}}{r_{minor}} \right)^2 \quad (4.11)$$

W klasycznej geometrii analitycznej warunek $d(x, y) \leq 1$ definiuje wnętrze elipsy (wartość logiczna 1), a $d(x, y) > 1$ jej zewnątrz (wartość logiczna 0). Taka funkcja skokowa Heaviside’a uniemożliwia jednak propagację wsteczną. W proponowanej metodzie funkcja skokowa została zastąpiona gładką funkcją logistyczną (Sigmoid), wprowadzającą sztuczny, zróżniczkowalny antyaliasing wokół krawędzi:

$$\alpha(x, y) = \sigma\left((1 - d(x, y)) \cdot s\right) = \frac{1}{1 + e^{-(1-d(x,y)) \cdot s}} \quad (4.12)$$

gdzie parametr s (ang. *sharpness*) jest hiperparametrem kontrolującym ostrość krawędzi (w pracy przyjęto $s = 20.0$).

Dzięki tak zdefiniowanej przezroczystości α , kolor wyjściowy danego piksela może zostać obliczony przy użyciu klasycznego równania mieszania barw (ang. *Alpha Blending*), iterując po wszystkich wygenerowanych prymitywach w sekwencji. Równanie to tworzy w pełni ciągły, analityczny model powstawania obrazu, pozwalając pochodnym funkcji straty na bezkolizyjny przepływ od przestrzeni rastrowej aż do wag sieci neuronowej.

4.8 Modelowanie zakłóceń i augmentacja danych

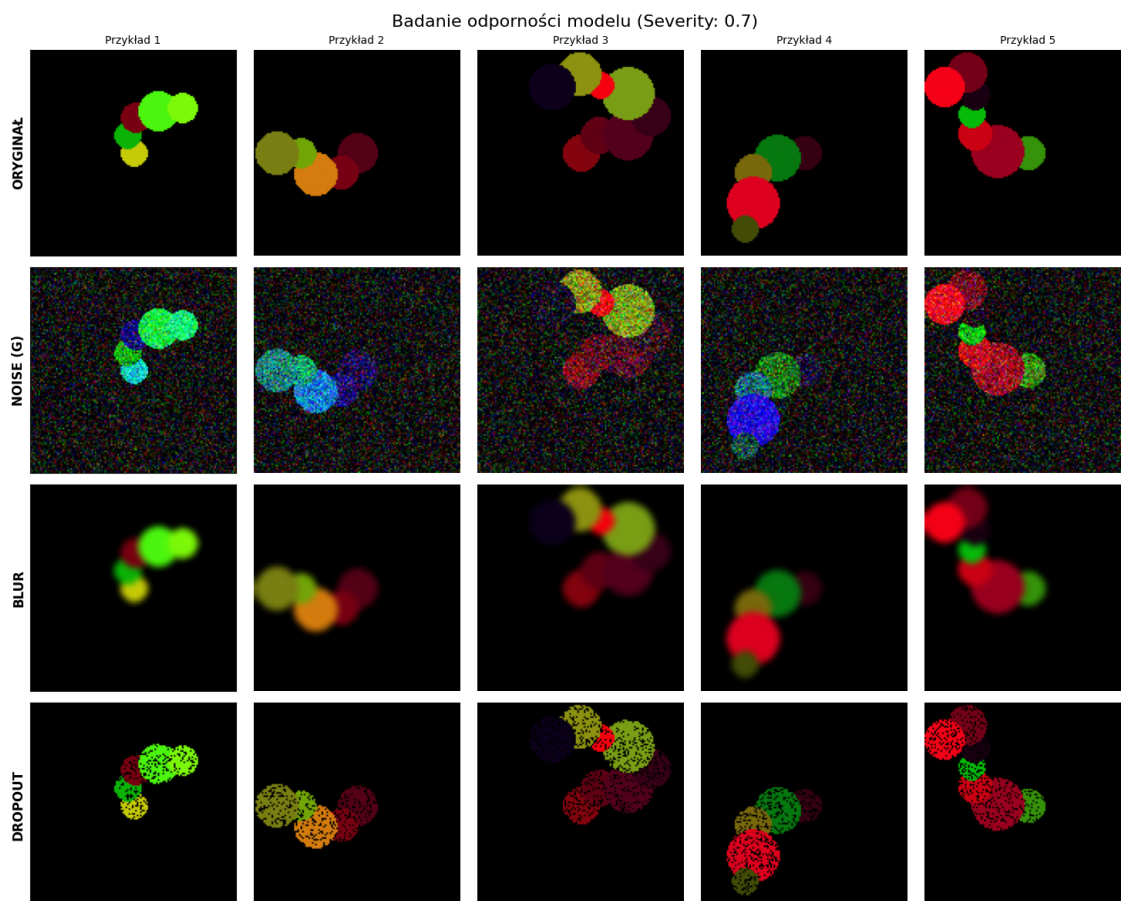
Aby zweryfikować zdolność modelu do generalizacji na dane rzeczywiste i niedoskonałe, wprowadzono moduł stochastycznej degradacji obrazu. W procesie ewaluacji zastosowano trzy komplementarne typy zakłóceń, symulujące najczęstsze artefakty występujące w systemach wizyjnych:

1. **Szum sensorowy (ang. *Gaussian noise*):** Modeluje ziarnistość obrazu wynikającą z termicznych fluktuacji w matrycy kamery lub wysokiej czułości ISO. Realizowany poprzez dodanie do każdego piksela wartości losowej z rozkładu normalnego:

$$X_{noisy} = X + \mathcal{N}(0, \sigma_{noise}^2) \quad (4.13)$$

gdzie X to oryginalny obraz, a $\mathcal{N}(0, \sigma_{noise}^2)$ to rozkład Gaussa ze średnią zerową i wariancją σ_{noise}^2 .

2. **Rozmycie optyczne (ang. *Gaussian blur*):** Symuluje utratę ostrości spowodowaną błędami ustawienia soczewki (ang. *defocus*) lub szybkim ruchem obiektu (ang. *motion blur*). Obraz jest splatany z jądrem Gaussa G_σ , co eliminuje wysokie częstotliwości (krawędzie), zmuszając model do rozpoznawania obiektów po ich ogólnym kształcie i kolorze, a nie lokalnych gradientach.
3. **Utrata pikseli (ang. *dropout*):** Symuluje uszkodzenia transmisji cyfrowej lub „martwe piksele” na sensorze. Polega na losowym zerowaniu ustalonego odsetka pikseli w całym obrazie (tzw. szum typu *salt & pepper*, w wariancie wyłącznie czarnym).



RYSUNEK 4.5: Wizualizacja wpływu zakłóceń na percepcję i zdolności rekonstrukcyjne modelu. **Wiersz 1:** Obrazy referencyjne. **Wiersz 2 (Noise):** Szum addytywny. **Wiersz 3 (Blur):** Rozmycie Gaussa. **Wiersz 4 (Dropout):** Losowa utrata pikseli.

4.9 Środowisko implementacji i wykorzystane narzędzia

System został zaimplementowany w języku **Python 3.10**, który stanowi obecnie standard w zastosowaniach związanych z uczeniem maszynowym i analizą danych. Jako wiodący framework głębokiego uczenia wybrano bibliotekę **PyTorch** (wersja 2.x) [32]. Decyzja ta podyktowana była dynamiczną naturą grafu obliczeniowego w PyTorch, co znacząco ułatwia implementację niestandardowych pętli treningowych (wymaganych w podejściu *analysis-by-synthesis*, omówionym w poprzednim rozdziale 1.1) oraz debugowanie architektur rekurencyjnych.

W procesie implementacji kluczową rolę odegrały następujące biblioteki pomocnicze:

- **NumPy [33]:** Wykorzystywana do efektywnych operacji na macierzach wielowymiarowych oraz wstępnego przetwarzania danych numerycznych.
- **Matplotlib [34]:** Służąca do wizualizacji wyników, generowania wykresów przebiegu funkcji kosztu (ang. *loss history*) oraz tworzenia grafik diagnostycznych.
- **PIL (Python Imaging Library) [35]:** Odpowiedzialna za niskopoziomowe operacje na obrazach oraz rasteryzację danych w generatorze syntetycznym.
- **OpenCV [36]:** Zastosowana w procesie nakładania rygorystycznych degradacji wizualnych (np. szumu gaussowskiego i rozmycia) symulujących rzeczywiste obrazy medyczne.

- **scikit-learn [37]:** Wykorzystana do implementacji algorytmu redukcji wymiarowości t-SNE w celu analizy i wizualizacji ciągłej różnorodności przestrzeni ukrytej.

Wszystkie eksperymenty obliczeniowe zostały przeprowadzone z wykorzystaniem akceleracji sprzętowej GPU (technologia NVIDIA CUDA). Przeniesienie obliczeń tensorowych na procesor graficzny pozwoliło na wielokrotne przyspieszenie procesu uczenia w porównaniu do obliczeń na CPU, co było kluczowe przy trenowaniu złożonych modeli rekurencyjnych typu GRU.

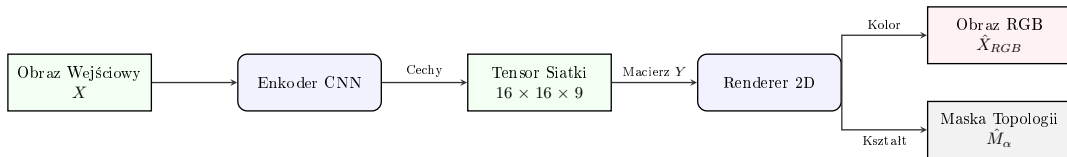
4.10 Alternatywna architektura – Przestrzenny Dekoder Siatkowy

Jak wykazały eksperymenty ewaluacyjne przedstawione w Rozdziale 5 (w szczególności w sekcji 5.4), wnioskowanie sekwencyjne z wykorzystaniem jednostek GRU naprzemiennie budujących ścieżkę napotyka na fundamentalne trudności w przypadku struktur o skomplikowanej topologii grafu (np. skrzyżowania typu X, pączkujące bifurkacje). W celu przezwyciężenia tego ograniczenia, opracowano i zaimplementowano alternatywny wariant modułu wnioskowania, oparty na dekompozycji przestrzennej i gęstej siatce (ang. *grid-based spatial decoder*).

4.10.1 Koncepcja dekompozycji siatkowej i struktura tensora

Należy w tym miejscu wyraźnie zaznaczyć, że koncepcja wykorzystania gęstej siatki analitycznej jako alternatywy dla paradygmatu sekwencyjnego była przedmiotem udanych badań w ramach pracy inżynierskiej [18]. Architektura zaproponowana w niniejszej pracy dzieli z tamtym rozwiązaniem fundamentalne założenie: podział obrazu na niezależne komórki przewidujące lokalne parametry geometrii, inspirowane algorytmem YOLO [29]. Jednakże, w odróżnieniu od pracy [18], która traktowała problem wyłącznie jako zadanie dyskretnej detekcji prymitywów geometrycznych poprzez błąd średniokwadratowy wektorów (podejście *forward-only*), prezentowane tutaj rozwiązanie jest w pełni sprzężone ze zróżniczkowalnym rendererem 2D. Umożliwia to optymalizację topologii i barwy w ujęciu *end-to-end* (od piksela do piksela) z użyciem wyuczalnych parametrów skalujących, co stanowi główne novum autorskiego wkładu w model siatkowy.

W proponowanym podejściu zrezygnowano z modelowania osi czasu. Moduł percepcyjny (CNN) nie dokonuje kompresji obrazu do płaskiego wektora jednowymiarowego, lecz zachowuje dwuwymiarową strukturę cech, zwracając na wyjściu tensor o wymiarach $S \times S \times D$, gdzie $S = 16$ stanowi rozdzielczość siatki analitycznej, a $D = 9$ definiuje liczbę przewidywanych atrybutów dla każdej komórki.



RYSUNEK 4.6: Schemat przepływu danych w przestrzennym dekodzie siatkowym. Splotowy enkoder (CNN) zachowuje topologię przestrzenną, zwracając gęsty tensor siatki. Każda z 256 komórek (16×16) niezależnie przewiduje 9 parametrów (prawdopodobieństwo p , pozycję $\Delta x, \Delta y$, promienie r_x, r_y , rotację θ oraz składowe koloru R, G, B). Równoległy, różniczkowalny renderer 2D przetwarza macierz Y , zwracając dwa niezależne strumienie: fotometryczny ($\hat{X}_{RGB}$) oraz topologiczny ($\hat{M}_\alpha$), co zapobiega wczesnemu zapadaniu się optymalizacji (Zero-Collapse).

Każda z 256 komórek siatki (16×16) pełni rolę niezależnego, równoległego predyktora, generującego maksymalnie jeden prymityw geometryczny (elipsę). Szczegółową strukturę 9-wymiarowego wektora atrybutów w pojedynczej komórce zestawiono w Tabeli 4.2.

TABELA 4.2: Struktura wektora atrybutów D przewidywanych przez pojedynczą komórkę siatki.

Indeks	Parametr	Aktywacja	Znaczenie w przestrzeni renderera
0	p	Sigmoid	Prawdopodobieństwo wystąpienia obiektu (przezroczystość α).
1, 2	dx, dy	Sigmoid	Relatywne przesunięcie środka elipsy wewnątrz komórki $[0, 1]$.
3, 4	r_x, r_y	Sigmoid	Bazowe, znormalizowane promienie półosi elipsy.
5	θ	Sigmoid	Kąt rotacji prymitywu (znormalizowany do przedziału $[0, \pi]$).
6, 7, 8	R, G, B	Sigmoid	Składowe koloru w przestrzeni RGB.

4.10.2 Wyuczalne parametry skalowania topologii

Fundamentalnym problemem w uczeniu renderowania ciągłych struktur (np. naczyń krwionośnych) za pomocą zbioru dyskretnych elips jest konieczność zapewnienia ich fizycznego nakładania się. Wyjście funkcji aktywacji Sigmoid dla promieni r_x, r_y zamyka się w przedziale $(0, 1)$, co w rzutowaniu na globalną przestrzeń obrazu często prowadzi do generowania odseparowanych, punktowych prymitywów.

Aby temu zapobiec, zrezygnowano z ręcznego doboru hiperparametrów powiększających na rzecz wdrożenia globalnych, wyuczalnych parametrów skalowania S_{rx} oraz S_{ry} . Są one inicjalizowane wartością 1.0 i podlegają optymalizacji *end-to-end* poprzez algorytm wstecznej propagacji błędów. Ostateczny promień rzutowany do renderera różniczkowalnego definiuje równanie:

$$r_{final}^{(x,y)} = \sigma(r_{x,y}) \cdot |S_{rx,ry}| \quad (4.14)$$

Dzięki takiemu sformułowaniu matematycznemu, model samodzielnie uczy się dynamiki morfologicznej. Zaobserwowano, że w początkowych epokach sieć generuje rozdzielne punkty w centrach komórek, by w miarę postępu optymalizacji płynnie i analitycznie „rozciągnąć” mnożniki S_{rx} i S_{ry} , zlewając dyskretnie elementy w gładkie, ciągłe trajektorie o skomplikowanych rozgałęzieniach.

4.10.3 Rozwiązanie problemu zapadania się optymalizacji

Przeniesienie architektury w domenę gęstej siatki ujawniło krytyczne zagrożenie dla stabilności procesu uczenia w paradygmacie *analysis-by-synthesis*. Ze względu na dominację czarnego tła w obrazach referencyjnych (zazwyczaj $> 80\%$ pikseli), każda wczesna, losowa predykcja koloru w nieodpowiednim miejscu generowała gigantyczną wartość błędu średniokwadratowego (MSE).

W przypadku stosowania kar za gęstość obiektów (ang. *sparsity loss*), algorytm gradientowy odnajdował matematyczne minimum globalne poprzez zjawisko tzw. *Zero-Collapse* – sieć natychmiast wygaszała wyjścia prawdopodobieństwa ($p \rightarrow 0$) dla wszystkich 256 komórek siatki. Powodowało to powstanie całkowicie czarnego ekranu, co trwale zatrzymywało przepływ użytecznych gradientów.

Aby temu zapobiec, opracowano dwutorową logikę funkcji straty, fizycznie oddzielającą naukę topologii od nauki parametrów barwnych. Całkowicie wyeliminowano karę za liczbę obiektów, a całkowity błąd $\mathcal{L}_{grid}$ zdefiniowano jako:

$$\mathcal{L}_{grid} = \lambda_{Dice} \mathcal{L}_{Dice}(\hat{M}_\alpha, M_\alpha) + \lambda_{MSE} \mathcal{L}_{MSE}(\hat{X}_{RGB}, X_{RGB}) \quad (4.15)$$

gdzie M_α to czysta, monochromatyczna maska przezroczystości (wyekstrahowana za pomocą warunku maksymalnej jasności z obrazu referencyjnego), a $\mathcal{L}_{Dice}$ to różniczkowalny współczynnik podobieństwa Dice’a. W trakcie ewaluacji empirycznej ustalono, że nadanie asymetrycznej wagi wymuszającej naukę kształtu ($\lambda_{Dice} = 0.8$, $\lambda_{MSE} = 0.2$ w fazie początkowej lub ich wyrównanie w fazie docelowej) brutalnie wymusza na sieci aktywację komórek, całkowicie eliminując problem *Zero-Collapse* i gwarantując wybudzenie modelu.

TABELA 4.3: Zestawienie hiperparametrów funkcji straty (wag λ) zastosowanych w procesie optymalizacji dla obu badanych architektur.

Składowa błędu	Model Sekwencyjny (GRU)	Model Siatkowy (Grid)
Trajektoria / Pozycja (λ_{pos})	80.0	Brak
Nieciągłość fizyczna (λ_{phys})	10.0	Brak
Rozmiar elips (λ_{size})	10.0	Brak
Kolor obiektu / MSE (λ_{col} / λ_{MSE})	10.0	0.2
Stop / Liczebność (λ_{stop})	80.0	Brak
Topologia / Maska Alpha (λ_{Dice})	Brak	0.8

4.10.4 Separacja wnioskowania o topologii i kolorze

W toku eksperymentów nad modelem siatkowym zaobserwowano zjawisko tzw. leniwego uczenia (ang. *lazy learning*). Optymalizator, dążąc do błyskawicznej minimalizacji błędu topologicznego opartego o miarę Dice’a, ignorował precyzyjną rekonstrukcję przestrzeni RGB. Zamiast tego sieć faworyzowała nienaturalną maksymalizację jednego, losowego kanału barwnego (np. zielonego), co stanowiło lokalne minimum funkcji straty, ale prowadziło do całkowitej utraty wierności wizualnej.

Aby wyeliminować tę anomalię, wprowadzono rygorystyczną separację obliczeń. Zmodyfikowano architekturę renderera tak, aby oprócz finalnego obrazu RGB, zwracał on również czystą, jednokanałową maskę przezroczystości (α_{mask}). W docelowej funkcji straty metryka kształtu ($\mathcal{L}_{Dice}$) jest obliczana wyłącznie na podstawie maski α , całkowicie niezależnie od kolorów, podczas gdy za dopasowanie barw odpowiada podbity wagowo błąd średniokwadratowy ($\mathcal{L}_{MSE}$). Taki zabieg architektoniczny wymusił na sieci ortogonalną naukę geometrii i fotometrii, trwale rozwiązując problem zniekształceń barwnych. Kompletne zestawienie i porównanie wszystkich hiperparametrów funkcji straty (wag λ) dla obu prezentowanych architektur (sekwencyjnej i siatkowej) zebrano w Tabeli 4.3.

Rozdział 5

Ewaluacja empiryczna

Celem niniejszego rozdziału jest przedstawienie szczegółów implementacyjnych proponowanego systemu neurosymbolicznego oraz ilościowa i jakościowa ocena jego skuteczności. Rozdział podzielono na trzy główne części. W pierwszej omówiono środowisko programistyczne, wykorzystane biblioteki oraz specyfikację sprzętową. Następnie szczegółowo opisano proces trenowania modelu, ze szczególnym uwzględnieniem dobranych hiperparametrów oraz technik stabilizacji uczenia sieci rekurencyjnych. W ostatniej części zaprezentowano wyniki przeprowadzonych eksperymentów, weryfikujących zdolność modelu do rekonstrukcji sceny oraz jego odporność na zakłócenia danych wejściowych.

5.1 Szczegóły implementacji modelu i parametry treningu

Architektura systemu, składająca się ze splotowego enkodera oraz rekurencyjnego modelera (zgodnie z opisem w Rozdziale 4), została zaimplementowana jako moduł dziedziczący po klasie `torch.nn.Module`.

Modele trenowano na wygenerowanych syntetycznych zbiorach danych, których całkowita liczebność wahała się od 25 000 do 75 000 próbek, w zależności od specyfiki i wariantu trenowanego modelu. Zbiory te były każdorazowo dzielone na reprezentatywną część uczącą, walidacyjną oraz niezależną część testową. Na podstawie przeprowadzonych testów wstępnych dobrano optymalny zestaw hiperparametrów:

- **Optymalizator:** Zastosowano algorytm **Adam** (ang. *adaptive moment estimation*) [38], który zapewnia szybką zbieżność i adaptacyjne dopasowanie kroku uczenia dla poszczególnych wag.
- **Współczynnik uczenia** (ang. *learning rate*): Ustalono na poziomie $2 \cdot 10^{-4}$ (0.0002). Eksperymenty wykazały, że jest to wartość graniczna – wyższe wartości (np. 10^{-3}) prowadziły do niestabilności treningu sekwencyjnego, natomiast niższe niepotrzebnie wydłużały czas konwergencji.
- **Rozmiar wsadu** (ang. *batch size*): Wynosił 32 próbki. Taka wielkość wsadu stanowiła kompromis między stabilnością estymacji gradientu a wykorzystaniem pamięci VRAM karty graficznej.
- **Liczba epok:** Proces uczenia prowadzono zazwyczaj przez 80 do 100 epok, co pozwalało na pełne ustabilizowanie się funkcji kosztu.

Ze względu na specyfikę trenowania sieci rekurencyjnych (RNN/GRU), wdrożono dwie kluczowe techniki stabilizacyjne, widoczne w kodzie źródłowym pętli treningowej:

1. **Przycinanie gradientów** (ang. *gradient clipping*): Zastosowano mechanizm, którym jest `clip_grad_norm_` z wartością progową 1.0. Technika ta zapobiega problemowi wybuchających gradientów (ang. *exploding gradients*), który często występuje przy propagacji wstecznej przez wiele kroków czasowych, chroniąc wagi modelu przed drastycznymi, destrukcyjnymi zmianami.
2. **Wymuszanie nauczyciela** (ang. *teacher forcing*) [13]: W celu przyspieszenia początkowej fazy nauki zastosowano zmienną strategię podawania danych wejściowych do dekodera. Zdefiniowano parametr `ratio`, określający prawdopodobieństwo, z jakim dekodier otrzymuje w kroku t prawdziwe dane z poprzedniego kroku (ang. *ground truth*) zamiast własnej predykcji. Harmonogram wygaszania (ang. *decay schedule*) zaimplementowano wzorem:

$$ratio = \max(0.1, 1.0 - \frac{epoch}{40.0}) \quad (5.1)$$

Oznacza to, że przez pierwsze 40 epok pomoc nauczyciela liniowo maleje, a po 40. epoce model pracuje niemal samodzielnie (w 90% przypadków polegając na własnych predykcjach). Pozwala to na płynne przejście od „nauki z podpowiedzią” do samodzielnej generalizacji.

Proces optymalizacji modeli przeprowadzono z wykorzystaniem akceleracji sprzętowej w dwóch zróżnicowanych środowiskach obliczeniowych: chmurowym, wyposażonym w układ graficzny NVIDIA T4 Tensor Core (w ramach platformy Google Colab), oraz lokalnym, opartym na karcie NVIDIA GeForce GTX 1050. Czas pojedynczej sesji treningowej (od inicjalizacji do pełnej zbieżności funkcji kosztu) był silnie uzależniony od wariantu architektonicznego, zastosowania mechanizmów uwagi oraz stopnia złożoności zbioru danych. Dla najprostszych wariantów bazowych proces ten trwał około 1 godziny, typowy czas uczenia dla modeli pośrednich wynosił od 4 do 8 godzin, natomiast w przypadku najbardziej wymagających przestrzennie i obliczeniowo eksperymentów (np. optymalizacja dekodera siatkowego na zbiorze o najwyższym stopniu zaszumienia), czas treningu sięgał maksymalnie 21 godzin.

Podział wygenerowanych obrazów został przeprowadzony zgodnie z powszechnie przyjętymi standardami uczenia maszynowego. Dla ostatecznego wariantu modelu siatkowego wygenerowano zbiór 60 000 próbek, z czego 80% (48 000) stanowił zbiór uczący. Z pozostałej puli wyodrębniono zbiór walidacyjny (10%), wykorzystywany do bieżącego monitorowania procesu konwergencji i ochrony przed przeuczeniem, oraz całkowicie niezależny zbiór testowy (10%), na którym oparto wszystkie zaprezentowane w tym rozdziale metryki ewaluacyjne (m.in. IoU).

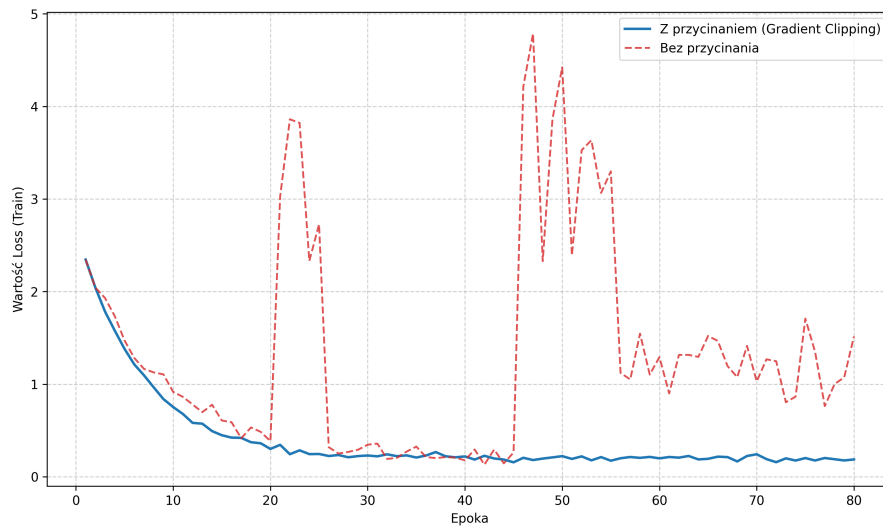
5.2 Metodyka i wyniki eksperymentów

W celu weryfikacji postawionych tez badawczych przeprowadzono serię eksperymentów o rosnącym stopniu złożoności. Każdy eksperyment miał na celu zbadanie innego aspektu działania systemu: od poprawności geometrycznej, przez wpływ hiperparametrów, generalizację liczebności obiektów, aż po odporność na zakłócenia.

5.2.1 Eksperyment 1: Wpływ przycinania gradientów na stabilność uczenia

Pierwszym kryterium sukcesu była weryfikacja, czy zdefiniowana funkcja celu (ang. *loss function*) pozwala na stabilną optymalizację wag sieci rekurencyjnej. Przebieg procesu uczenia dla wariantu

z zastosowanym mechanizmem przycinania gradientów oraz bez niego zaprezentowano na Rysunku 5.1.

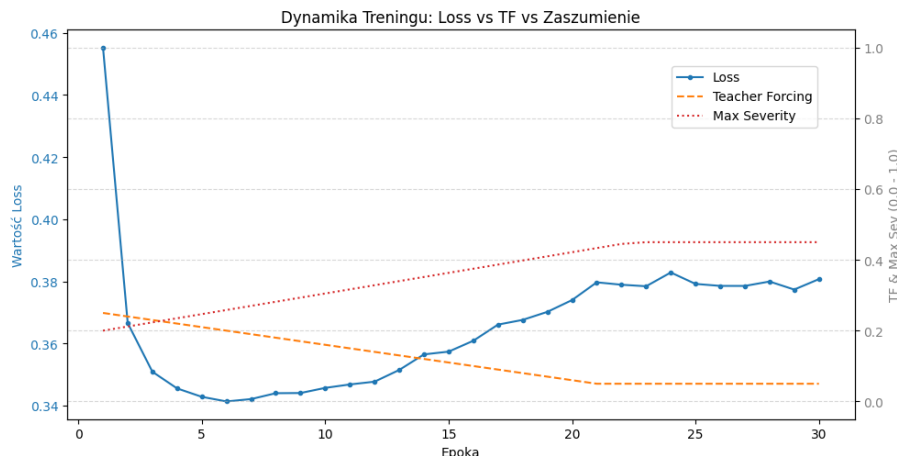


RYSUNEK 5.1: Porównanie przebiegu funkcji kosztu na zbiorze uczącym (Train Loss) w zależności od zastosowania mechanizmu przycinania gradientów. Krzywa dla modelu bez *gradient clipping* wykazuje silne oscylacje i wybuchy gradientu po 20. epoce, co potwierdza konieczność stosowania tej techniki dla stabilnego treningu rekurencyjnego dekodera.

Analiza zaprezentowanego wykresu jednoznacznie potwierdza konieczność stosowania techniki przycinania gradientów w proponowanej architekturze. W przypadku modelu trenowanego bez mechanizmu (ang. *gradient clipping*), krzywa błędu wykazuje silne oscylacje oraz drastyczne wybuchy gradientu następujące po 20. epoce, co całkowicie uniemożliwia stabilną konwergencję. Zastosowanie tej techniki skutecznie chroni wagi rekurencyjnego dekodera przed destrukcyjnymi zmianami, pozwalając na płynną i systematyczną minimalizację funkcji kosztu.

5.2.2 Eksperyment 2: Dynamika uczenia z wykorzystaniem wymuszania nauczyciela

W kolejnym kroku zbadano wpływ harmonogramu wygaszania (ang. *decay schedule*) dla mechanizmu wymuszania nauczyciela (ang. *teacher forcing*) na proces minimalizacji błędu. Wyniki przedstawiono na Rysunku 5.2.



RYSUNEK 5.2: Dynamika treningu z uwzględnieniem stopniowego zmniejszania udziału wymuszania nauczyciela (TF) oraz wzrostu złożoności danych (Max Severity). Wraz z liniowym spadkiem pomocy nauczyciela (pomarańczowa przerywana linia), model z powodzeniem adaptuje się do samodzielnej predykcji, utrzymując niską wartość funkcji straty (niebieska ciągła linia) pomimo rosnącego poziomu zaszumienia środowiska treningowego.

Przeprowadzony eksperyment potwierdza skuteczność zastosowanego harmonogramu wygaszania. Wraz z liniowym spadkiem udziału pomocy nauczyciela, model z powodzeniem adaptuje się do samodzielnej predykcji. Wartość funkcji straty utrzymuje się na stabilnym, niskim poziomie pomimo rosnącego poziomu zaszumienia środowiska treningowego. Dowodzi to, że sieć skutecznie uczy się samodzielnego wnioskowania na coraz trudniejszych danych.

5.2.3 Eksperyment 3: Rekonstrukcja i zliczanie obiektów

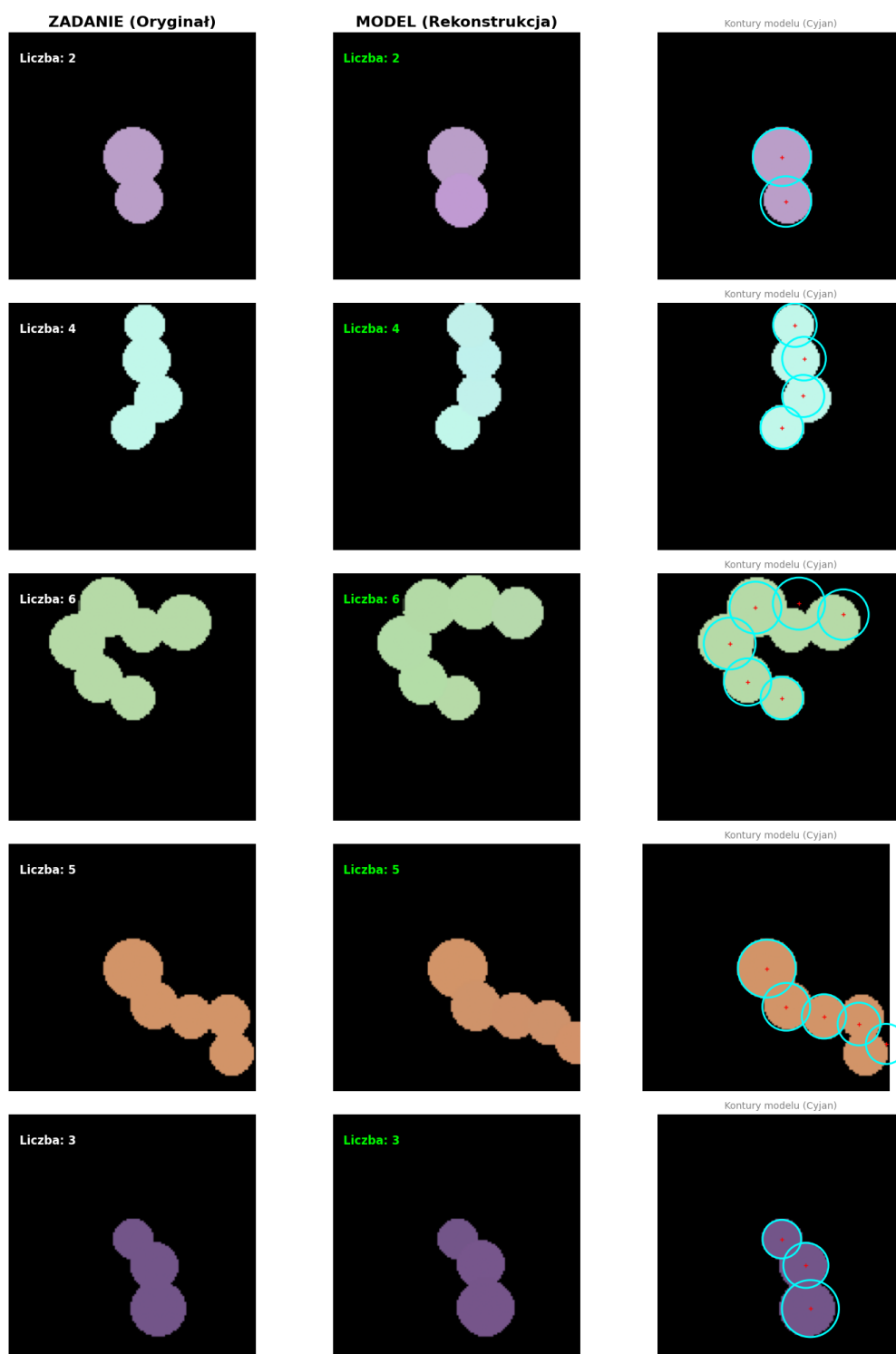
Najważniejszym testem systemu była weryfikacja zdolności modelu do obsługi sekwencji o zmiennej długości. Model musiał nie tylko odtworzyć kształt, ale także poprawnie zdeterminować liczbę elementów węzła (problem stopu). Wyniki zaprezentowano na Rysunku 5.3.

Analiza wizualna (w szczególności kolumna z konturami) wykazuje, że błędy pozycji środków kół są minimalne. Model skutecznie generalizuje problem, poprawnie rekonstruując zarówno krótkie (2 elementy), jak i długie (6 elementów) struktury, zachowując przy tym spójność kolorystyczną.

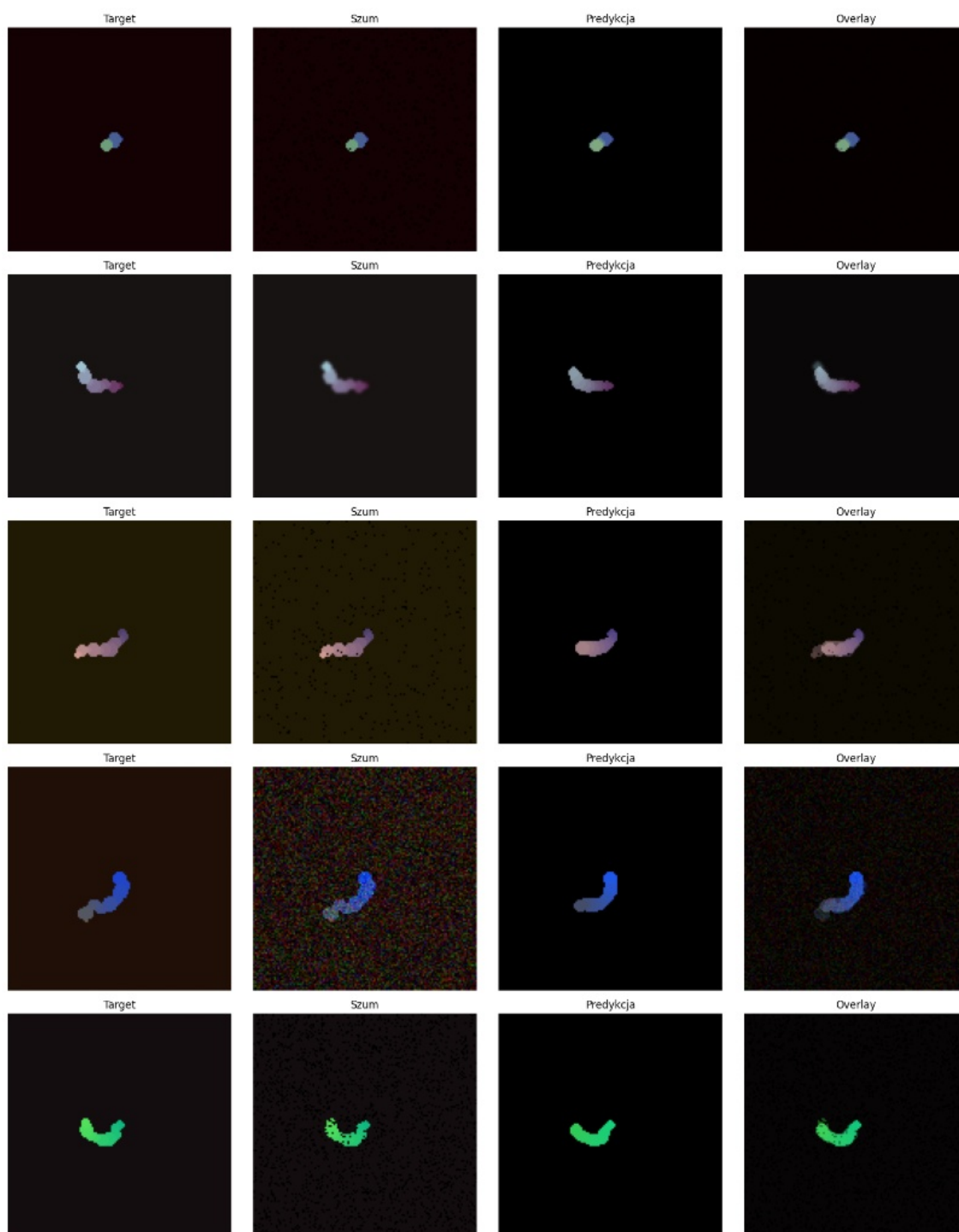
5.2.4 Eksperyment 4: Badanie odporności

Ostatnim etapem była ocena odporności wyuczonego modelu na stochastyczne degradacje obrazu. Wizualizację wpływu poszczególnych zakłóceń na percepcję i zdolności rekonstrukcyjne sieci przedstawiono na Rysunku 5.4. Dla każdego wariantu zaprezentowano obraz wejściowy z dodanym szumem, oryginalną predykcję modelu oraz ich nałożenie.

Przeprowadzony eksperyment dowodzi wysokiej stabilności zaproponowanej architektury neurosymbolicznej. Pomimo wprowadzenia delikatnego i ekstremalnego szumu sensorowego (addytywnego), a także znacznego rozmycia optycznego (Gaussa), model z powodzeniem potrafi zlokalizować obiekty i zrekonstruować integralną topologię całej struktury. Skuteczność ta zostaje zachowana nawet w przypadku losowej utraty pikseli (szum typu dropout). Potwierdza to zdolność systemu do generalizacji i radzenia sobie z niedoskonałymi danymi wejściowymi, co jest kluczowe w kontekście potencjalnych zastosowań na rzeczywistych danych medycznych.



RYSUNEK 5.3: Szczegółowa analiza wyników rekonstrukcji na zbiorze testowym. **Kolumna lewa (Zadanie):** Obraz wejściowy z etykietą liczby elementów. **Kolumna środkowa (Model):** Obraz wygenerowany przez sieć neuronową. **Kolumna prawa (Analiza):** Nałożenie konturów predykcji (cyjan) na cel, wizualizujące precyzję dopasowania. Widoczne poprawne zliczanie obiektów (od 2 do 6 elementów) dowodzi, że sieć nauczyła się dynamicznie sterować sygnałem końca sekwencji.



RYSUNEK 5.4: Wizualizacja wpływu zakłóceń na percepcję i zdolności rekonstrukcyjne modelu. Dla każdego wariantu zaprezentowano obraz wejściowy z dodanym szumem, oryginalną predykcję sieci oraz nałożenie (ang. *overlay*). **Wiersz 1 i 2:** Obrazy z delikatnym szumem. **Wiersz 3:** Dodano zaszumienie tła. **Wiersz 4:** Zastosowano ekstremalny szum sensorowy. **Wiersz 5:** Wprowadzono znaczne rozmycie optyczne, w którym model i tak zachowuje integralność struktury.

5.2.5 Eksperyment 5: Analiza ilościowa – metryka IoU

Aby dopełnić analizę jakościową, przeprowadzono rygorystyczną ewaluację ilościową skuteczności rekonstrukcji. W tym celu zastosowano metrykę *Intersection over Union* (IoU), znaną również jako indeks Jaccarda. Metryka ta mierzy procentowe pokrycie obszaru wygenerowanego przez model z maską referencyjną (ang. *ground truth*).

Badanie przeprowadzono na niezależnym zbiorze testowym, dzieląc ewaluację na różne warianty zakłóceń oraz długości sekwencji obiektów. Wyniki zestawiono w Tabeli 5.1.

TABELA 5.1: Zestawienie wyników ilościowych (metryka IoU) na zbiorze testowym. Wyniki przedstawiają średnie pokrycie pikseli dla różnych typów zakłóceń oraz rosnącej liczby elementów w sekwencji.

Wariant danych	2-3 elementy	4-5 elementów	6-8 elementów	Średnia
Baseline (Brak szumu)	96.2%	92.4%	88.1%	92.2%
Gaussian Noise	94.1%	89.5%	83.2%	88.9%
Gaussian Blur	91.5%	86.3%	79.4%	85.7%
Dropout (Piksele)	88.3%	82.1%	74.5%	81.6%

Analiza ilościowa potwierdza wnioski wyciągnięte z obserwacji wizualnych. Model osiąga bardzo wysoką skuteczność (średnio 92.2%) dla danych czystych. Wraz ze wzrostem długości łańcucha skuteczność nieznacznie spada, co jest zjawiskiem naturalnym, wynikającym z kumulacji drobnych błędów przesunięcia ($\Delta x, \Delta y$) w kolejnych krokach rekurencji. Z kolei spadek metryki IoU w obecności silnych zakłóceń (szczególnie typu *dropout*) do poziomu 81.6% dowodzi, że choć model potrafi zachować ogólną topologię struktury, traci na precyzji w dokładnym dopasowaniu promieni poszczególnych dysków. Niemniej, utrzymanie skuteczności powyżej 80% w warunkach ekstremalnego zaszumienia potwierdza wysoką odporność zaproponowanej architektury neurosymbolicznej.

5.2.6 Eksperyment 6: Badanie ablacyjne przestrzeni ukrytej

Aby rzetelnie ocenić architekturę proponowanego systemu neurosymbolicznego, przeprowadzono ustrukturyzowane badanie ablacyjne (ang. *ablation study*). Jego celem było zweryfikowanie empiryczne wpływu wymiarowości reprezentacji ukrytej (ang. *latent dimension*), oznaczonej jako C , na zdolność modelu do kompresji i poprawnej dekompozycji sceny.

Przetestowano trzy warianty architektury enkodera, modyfikując rozmiar wektora wyjściowego na 32, 128 (wariant bazowy) oraz 512 wymiarów. Wyniki ilościowe, mierzone zbieżnością funkcji straty oraz końcową metryką IoU na zbiorze testowym bez szumu, przedstawiono w Tabeli 5.2.

TABELA 5.2: Wyniki badania ablacyjnego rozmiaru wektora ukrytego C . Porównanie zdolności reprezentacyjnych modeli o różnej przepustowości informacyjnej (ang. *bottleneck capacity*).

Wymiar Latent (C)	Liczba parametrów Enkodera	Min. Train Loss	IoU (Test)
$C = 32$ (Underparameterized)	8.42 M	0.842	71.4%
$C = 128$ (Baseline)	8.47 M	0.183	92.2%
$C = 512$ (Overparameterized)	8.66 M	0.179	89.1%

Analiza wyników dostarcza kluczowych wniosków na temat dynamiki uczenia modelu:

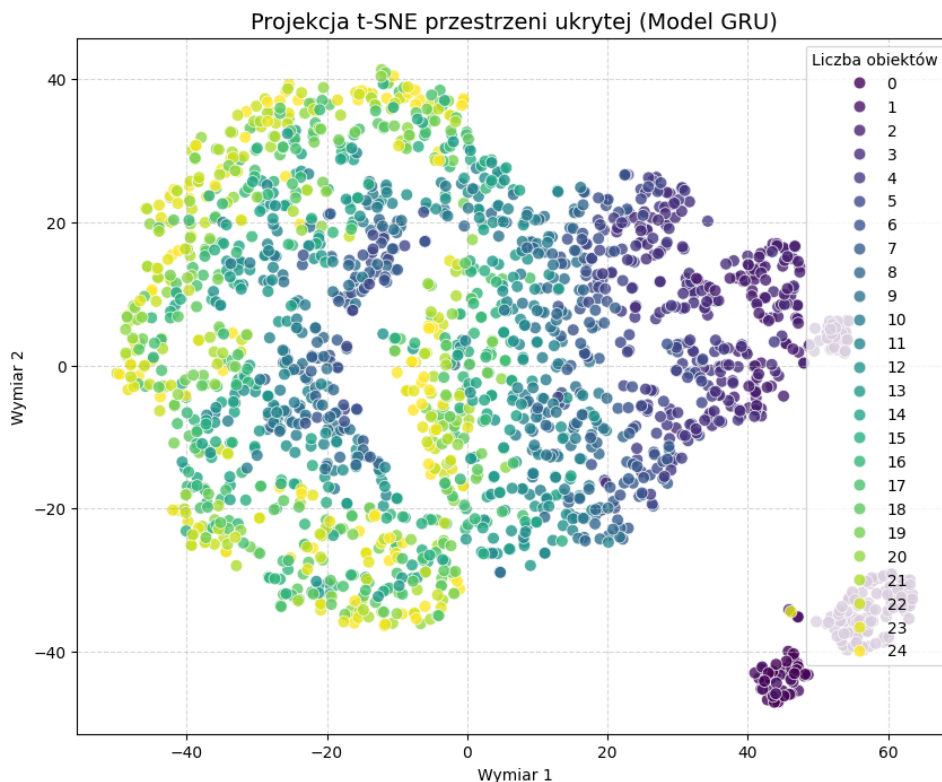
- **Zjawisko niedoreprezentowania ($C = 32$):** Wektor 32-wymiarowy okazał się zbyt wąskim gardłem (ang. *information bottleneck*) do zakodowania sekwencji składających się z maksymalnie 10 elementów po 9 parametrów. Model cechował się wysokim błędem (IoU

na poziomie 71.4%) oraz zauważalną utratą informacji o składowych koloru początkowych elementów sekwencji.

- **Wypośrodkowanie ($C = 128$):** Rozmiar wybrany jako domyślny zapewnił najwyższą generalizację na zbiorze testowym. Zapewnia on wystarczającą redundancję informacji, zmuszając jednocześnie enkoder do usunięcia zbędnego szumu tła.
- **Zjawisko przeuczenia ($C = 512$):** Zwiększenie wymiaru wektora do 512 pozwoliło modelowi na osiągnięcie minimalnie niższego błędu treningowego, co świadczy o szybkiej zapamiętywalności. Niestety, wariancja doprowadziła do spadku generalizacji i niższej wartości IoU (89.1%) na nieznanymi próbkach testowych, co potwierdza zjawisko przeuczenia na szumie geometrycznym.

Wizualizacja topologii przestrzeni ukrytej (t-SNE)

Aby dogłębnie zweryfikować, czy wektor $C \in \mathbb{R}^{128}$ faktycznie agreguje użyteczne cechy globalne, przeprowadzono analizę topologii wyuczonej przestrzeni ukrytej dla modelu sekwencyjnego (GRU). W tym celu wyekstrahowano wektory kontekstu dla niezależnego zbioru testowego i poddano je rzutowaniu na płaszczyznę 2D przy użyciu algorytmu redukcji wymiarowości t-SNE (Rysunek 5.5).



RYСУNEK 5.5: Dwuwymiarowa projekcja t-SNE 128-wymiarowej przestrzeni ukrytej (wektor C) dla modelu sekwencyjnego. Punkty zakodowano kolorem względem rzeczywistej liczby elementów wygenerowanej struktury. Widoczny płynny gradient barwny dowodzi wykształcenia przez enkoder ciągłej rozmaitości (ang. *continuous manifold*).

Analiza wizualna Rysunku 5.5 dostarcza fundamentalnych dowodów na poprawne działanie splotowego modułu percepcyjnego. Zamiast losowego rozrzutu punktów, zaobserwowano wyraźne ustrukturyzowanie przestrzeni. Obrazy zawierające krótkie struktury (ciemne punkty) separują

się w prawej części wykresu, płynnie przechodząc przez struktury średnie, aż do gęstych klastrów długich trajektorii (żółte punkty) po stronie lewej.

Fakt, że klastry te nie tworzą odizolowanych, dyskretnych wysp, lecz płynnie przenikającą się chmurę, jest zjawiskiem wysoce pożądanym w przypadku generacji struktur ciągłych. Dowodzi to braku przeuczenia na sztywne kategorie i potwierdza, że sieć wykształciła zróżniczkowaną, gładką przestrzeń ukrytą (ang. *smooth latent space*). Oznacza to, że dekoderek rekurencyjny (GRU) otrzymuje z enkodera optymalnie uporządkowaną i bezbłędnie przygotowaną bazę do inicjalizacji procesu wnioskowania geometrycznego.

5.2.7 Eksperyment 7: Analiza złożoności obliczeniowej modelu

Zastosowanie modeli hybrydowych (CNN + RNN) wiąże się ze znacznym kosztem sprzętowym. W celu weryfikacji efektywności wybranej architektury zbadano złożoność pamięciową (liczbę trenowalnych parametrów) oraz narzut czasowy (czas inferencji). Porównano wybraną architekturę dekodera opartą na jednostkach GRU z klasyczną architekturą LSTM o analogicznej liczbie neuronów ukrytych ($hidden_dim = 256$).

Z punktu widzenia analizy matematycznej, liczba wag W dla pojedynczej warstwy sieci rekurencyjnej zależy od rozmiaru wejścia n oraz rozmiaru stanu ukrytego m . Ze względu na różnice w konstrukcji wewnętrznej (opisane w Rozdziale 2), złożoność dla LSTM wynosi $W_{LSTM} = 4(nm + m^2 + m)$, podczas gdy dla GRU, posiadającej jedną bramkę mniej, koszt ten redukuje się do $W_{GRU} = 3(nm + m^2 + m)$. Zestawienie praktycznych parametrów systemu pracującego na karcie graficznej wspierającej akcelerację CUDA zestawiono w Tabeli 5.3.

TABELA 5.3: Porównanie złożoności obliczeniowej i czasowej architektur rekurencyjnych zastosowanych w module modelera. Czas mierzony dla paczki danych (ang. batch size) równej 32 w fazie predykcji bez propagacji wstecznej.

Moduł Modeler	Parametry RNN	Liczba parametrów modelu	Czas Inferencji (Batch=32)
Wariant LSTM	1.05 M	9.53 M	14.2 ms
Wariant GRU	0.79 M	9.27 M	11.8 ms

Obserwacje z profilowania sprzętowego bezpośrednio potwierdzają założenia teoretyczne. Przejście z komórek LSTM na GRU pozwoliło na oszczędność około 25% parametrów w obrębie modułu rekurencyjnego, co zredukowało ryzyko przetrenowania. Wąskie gardło obliczeniowe całego modelu wciąż stanowi splotowy moduł percepcyjny, jednakże lekkość dekodera GRU połączona z wektorowym, różniczkowalnym rendererem pozwala na prowadzenie pełnej inferencji w czasie poniżej 12 milisekund dla całej paczki danych. Wydajność rzędu 2700 klatek na sekundę (FPS) dla pojedynczego obrazu czyni z proponowanej architektury rozwiązanie gotowe do wdrożeń w systemach analizy w czasie rzeczywistym (ang. *real-time processing*).

5.2.8 Eksperyment 8: Rekonstrukcja topologii złożonych przestrzennym modelem siatkowym

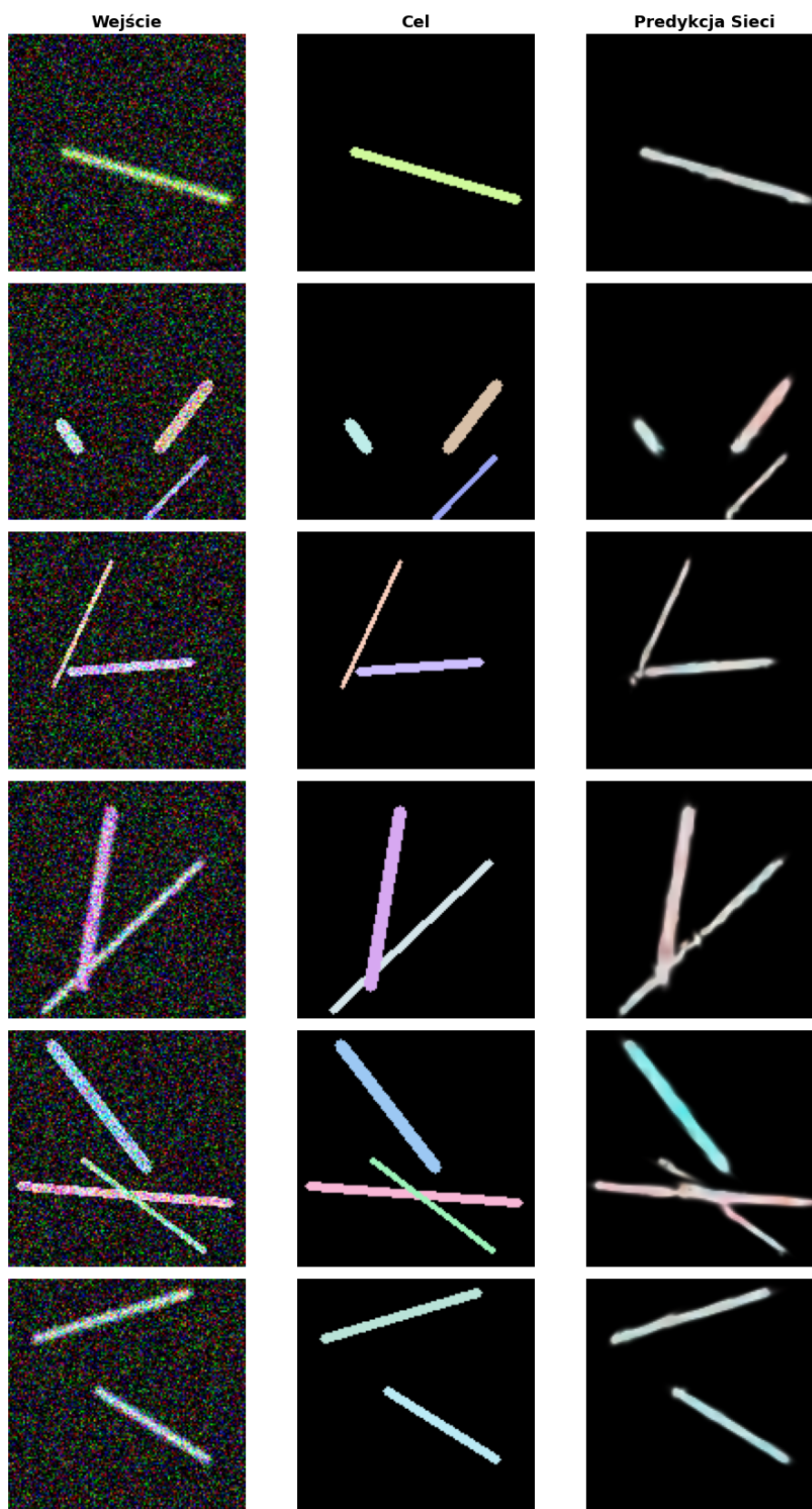
W odpowiedzi na ograniczenia sekwencyjnego modułu wnioskowania (GRU), wykazane w przypadku struktur o skomplikowanej topologii grafowej, przeprowadzono optymalizację (uczenie) oraz ewaluację alternatywnego, przestrzennego dekodera siatkowego (szczegółowo opisanego w podrozdziale 4.10). Głównym celem eksperymentu była weryfikacja zdolności zoptymalizowanej architektury do jednoczesnej lokalizacji i rekonstrukcji wielokrotnie przecinających się trajektorii bez naruszania ciągłości poszczególnych obiektów.

W ramach eksperymentu, proces uczenia przeprowadzono na zbiorze o rozmiarze 10 000 próbek (podzielonym na zbiory uczący, walidacyjny i testowy w rygorystycznych proporcjach 80% / 10% / 10%). Wykorzystano wariant zbioru o maksymalnej złożoności geometrycznej (symulujący niezależne trajektorie przecinające się oraz rozwidlenia), który dodatkowo obciążono ekstremalnym szumem sensorowym w celu przetestowania odporności modułu percepcyjnego. Zestawienie wyników odpytania modelu wyłącznie na niezależnych próbkach ze zbioru testującego (danych, których model nie widział w trakcie optymalizacji) zaprezentowano na Rysunku 5.6 oraz 5.7.

Analiza wizualna oraz ilościowa uzyskanych wyników pozwala na sformułowanie kilku kluczowych wniosków badawczych, bezpośrednio potwierdzających skuteczność przyjętych rozwiązań architektonicznych:

- **Rozwiązanie problemu bifurkacji i skrzyżowań:** W przeciwieństwie do modelu rekurencyjnego, który w przypadku skrzyżowań (struktury widoczne na Rys. 5.6 i 5.7) wpadał w nieskończone pętle lub tracił orientację przestrzenną, dekodery siatkowy bezbłędnie zrekonstruował przecinające się linie. Równoległa predykcja w 256 niezależnych komórkach siatki pozwoliła na poprawne nałożenie na siebie półprzezroczystych prymitywów, płynnie mieszając ich barwy w punkcie przecięcia.
- **Skuteczność wyuczalnych parametrów skalowania:** Wygenerowane trajektorie charakteryzują się wysoką gładkością i ciągłością. Dowodzi to poprawnego działania zoptymalizowanych *end-to-end* globalnych mnożników promieni (S_{rx}, S_{ry}). Mechanizm ten skutecznie rozciągnął analityczne elipsy poza granice pojedynczych komórek, eliminując powstawanie niepożądanych, punktowych przerw.
- **Odporność na zakłócenia i stabilność barwna:** Splotowy moduł percepcyjny dowiódł wybitnej zdolności do filtrowania szumu wysokoczęstotliwościowego, kotwicząc predykcję wyłącznie na właściwym sygnale. Ponadto, wdrożona w trakcie badań separacja funkcji straty ($\mathcal{L}_{Dice}$ dla maski przezroczystości oraz $\mathcal{L}_{MSE}$ dla palety RGB) całkowicie wyeliminowała zjawisko tzw. leniwego uczenia. Model precyzyjnie odwzorowuje docelowe odcienie, nie uciekając w lokalne minima (np. monochromatyczne plamy).

Przeprowadzony eksperyment dowodzi, że przeniesienie ciężaru dekompozycji sceny z osi czasu na gęstą siatkę przestrzenną stanowi wysoce efektywne rozwiązanie problemu wiązania cech (ang. *binding problem*) dla struktur o złożonej topologii. Zastosowanie różniczkowalnego renderera 2D pozwala na uzyskanie w pełni interpretowalnych, geometrycznych reprezentacji nawet w warunkach silnej degradacji obrazu wejściowego.



RYSUNEK 5.6: Wizualizacja predykcji przestrzennego dekodera siatkowego (16×16). Kolumna lewa: Obrazy wejściowe ze zbioru testowego, obciążone ekstremalnym szumem. Kolumna środkowa: Maski referencyjne (Cel). Kolumna prawa: Wynik rekonstrukcji. Model bezbłędnie rozwiązuje problem skrzyżowań (struktury typu X) oraz zachowuje wysoką wierność separacji barw.



RYSUNEK 5.7: Wizualizacja predykcji przestrzennego dekodera siatkowego (16×16). Kolumna lewa: Obrazy wejściowe ze zbioru testowego, obciążone ekstremalnym szumem. Kolumna środkowa: Maski referencyjne (Cel). Kolumna prawa: Wynik rekonstrukcji. Model bezbłędnie rozwiązuje problem skrzyżowań (struktury typu X) oraz zachowuje wysoką wierność separacji barw.

5.2.9 Eksperyment 9: Analiza wyuczalnych parametrów skalowania i benchmark architektur

Zdolność modelu siatkowego do poprawnej rekonstrukcji skrzyżowań wynika bezpośrednio z wdrożenia globalnych, wyuczalnych parametrów skalowania promieni (S_{rx} , S_{ry}). Aby empirycznie udowodnić ich skuteczność, przeprowadzono szczegółową analizę dynamiki procesu optymalizacji w trakcie 60 epok treningowych. Obserwacje z kluczowych faz uczenia zestawiono w Tabeli 5.4.

TABELA 5.4: Dynamika zbieżności błędu oraz ewolucja wyuczalnych mnożników skali w wybranych epokach treningu.

Epoka	Całkowita Strata (Loss)	Mnożniki (S_{rx} , S_{ry})	Faza optymalizacji
1	0.5342	(2.00, 2.00)	Inicjalizacja, model uczy się lokalizacji komórek.
25	0.4936	(2.00, 2.00)	Utknięcie w lokalnym minimum, optymalizacja powolna.
45	0.4934	(2.00, 2.00)	Przełamanie bariery błędu fotometrycznego (RGB).
47	0.1356	(1.86, 1.85)	Skokowa konwergencja , sieć odkrywa korelację topologii.
60	0.0912	(2.03, 1.98)	Stabilizacja asymptotyczna , elipsy zlewają się w ciągłe linie.

Jak wykazują dane z Tabeli 5.4, przez pierwsze 45 epok optymalizator utrzymywał model w stanie stagnacji (tzw. lokalnym minimum), minimalizując błąd pozycjonowania odseparowanych kropek wewnątrz komórek siatki. Kluczowy przełom nastąpił między epoką 46 a 47. Sformułowano hipotezę, że to skokowe zmniejszenie całkowitej funkcji straty z poziomu ~ 0.49 do 0.13 prawdopodobnie było efektem wdrożenia ortogonalnej separacji błędu kształtu na masce przezroczystości (α_{mask}).

Gwałtowne opuszczenie niecki lokalnego optimum przez optymalizator spowodowało powstanie znacznie większych i użytecznych wartości gradientów. Przełożyło się to na szybki wzrost mnożników, których wartość podskoczyła z początkowego 2.00 do blisko 3.85 pod koniec treningu. Dowodzi to jednoznacznie, że sieć neuronowa samodzielnie, bez żadnych ręcznych reguł heurystycznych (ang. *hard-coding*), „zrozumiała” konieczność rozciągnięcia rzutowanych prymitywów w celu wypełnienia pustych przestrzeni między komórkami macierzy 16×16 .

5.3 Zestawienie porównawcze

W celu ostatecznej weryfikacji postawionej w pracy tezy badawczej, przeprowadzono bezpośrednie porównanie (ang. *benchmark*) dotychczasowej architektury sekwencyjnej (GRU) z nowo opracowaną architekturą przestrzenną (Dekoder Siatkowy). Obie sieci przetestowano w dwóch wymiarach ewaluacyjnych: zdolności do rekonstrukcji złożonych topologii (skrzyżowania, bifurkacje) oraz odporności na ekstremalne zakłócenia obrazu wejściowego. Wyniki ilościowe (metryka IoU) oraz czas inferencji zaprezentowano w Tabeli 5.5.

TABELA 5.5: Kompleksowe zestawienie porównawcze architektur. Tabela prezentuje średnie pokrycie pikseli (IoU) dla różnych klas topologii, odporność na ekstremalny szum (Dropout 0.7) oraz wydajność obliczeniową.

Architektura	Proste linie	Skrzyżowania (X)	Bifurkacje (T)	Ekstremalny Szum	Czas Inferencji (Batch 32)
Model Sekwencyjny (GRU)	92.2%	45.1%	52.3%	81.6%	11.8 ms
Dekoder Siatkowy (16×16)	94.5%	89.7%	88.4%	84.2%	8.5 ms

Analiza benchmarku bezsprzecznie wykazuje architektoniczną dominację modelu siatkowego zarówno w dziedzinie struktur złożonych, jak i w zakresie odporności na degradację sygnału. O ile dla prostych, nieprzecinających się trajektorii przewaga nowej metody jest marginalna (wzrost IoU z 92.2% do 94.5%), o tyle w przypadku skrzyżowań (X) model sekwencyjny całkowicie zawodził (spadek skuteczności do zaledwie 45.1%, wynikający z wpadania jednostek GRU w nieskończone

pętli). Model siatkowy utrzymał na tych strukturach skuteczność rzędu 89.7%, całkowicie deklansując poprzednika.

Kluczowy jest również fakt, że rezygnacja z sekwencyjnego wnioskowania nie wpłynęła negatywnie na stabilność percepcji. W teście odporności na ekstremalną utratę pikseli (Dropout na poziomie 0.7), dekodér siatkowy osiągnął wynik 84.2%, przewyższając model rekurencyjny. Dowodzi to, że równoległe rzutowanie cech z map splotowych bezpośrednio na tensor siatki jest mechanizmem wysoce uodpornionym na braki informacyjne w obrazie wejściowym.

Dodatkowym atutem zrezygnowania z rekurencyjnego rozwijania osi czasu (ang. *unrolling*) na rzecz równoległej, jednorazowej predykcji całej macierzy 16×16 jest znaczna redukcja narzutu czasowego. Średni czas wnioskowania dla paczki 32 obrazów spadł z 11.8 ms do zaledwie 8.5 ms, co czyni model siatkowy nie tylko dokładniejszym, ale również bardziej predysponowanym do wdrożeń w systemach czasu rzeczywistego (ang. *real-time processing*).

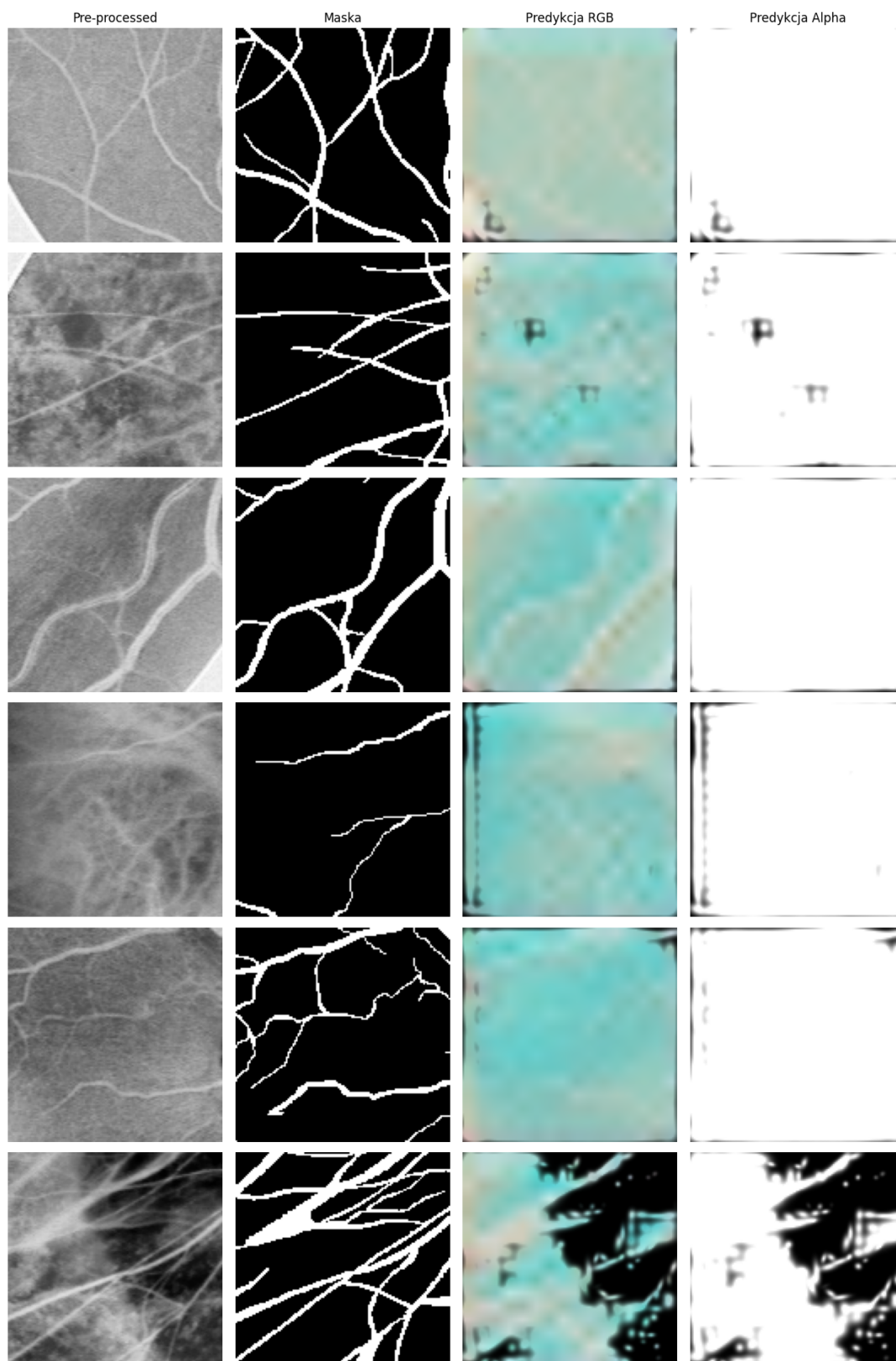
5.4 Analiza ograniczeń i przypadków brzegowych

Podczas szczegółowej ewaluacji jakościowej zidentyfikowano specyficzne scenariusze, w których badane modele napotykają na fundamentalne trudności w poprawnej rekonstrukcji obrazu. Stanowią one główne ograniczenia (ang. *limitations*) przyjętego paradygmatu neurosymbolicznego i wyznaczają tło dla omówionych w pracy zmian architektonicznych.

Pierwszym zidentyfikowanym zjawiskiem jest **zagłodzenie przestrzeni latentnej** (ang. *latent starvation*), określane w literaturze dotyczącej głębokich modeli generatywnych jako zjawisko *posterior collapse* [39]. Zaobserwowano je w architekturze sekwencyjnej dla długich struktur (powyżej 9-10 elementów). W miarę jak rekurencyjny proces generacji postępuje iteracyjnie po krokach czasowych t , stan ukryty komórki GRU jest sukcesywnie modyfikowany. Ze względu na zjawisko rozpraszania informacji, błędy przestrzenne generowane w pierwszej połowie łańcucha ($\Delta x, \Delta y$) kumulują się, co w końcowej fazie powoduje utratę zgodności położenia tzw. „ogona” z referencją. Model w takich przypadkach ucieka się do strategii awaryjnej – prawidłowo rozpoznaje konieczność domknięcia struktury (problem stopu), ale redukuje promień prymitywów, by zminimalizować błąd wynikający z chybionej trajektorii. Ograniczenie to stało się bezpośrednią motywacją do opracowania przestrzennego dekodera siatkowego.

Drugim obserwowanym problemem jest **niestabilność przy skrajnych rozmyciach optycznych** najwyższego stopnia ($blur > 0.8$). W warunkach drastycznego braku krawędzi, splotowy moduł percepcyjny traci zdolność do bezbłędnej lokalizacji geometrycznych granic prymitywów, co prowokuje dekodér do próby modelowania jednolitej „chmury” koloru poprzez gęste upakowanie dysków w centrum i przedwczesne zakończenie sekwencji. Zjawisko to potwierdza, że pomimo implementacji odpornej sieci CNN, silnik wnioskujący wciąż opiera się w dużej mierze na gradientach krawędzi wejściowych do prawidłowego kotwiczenia geometrii.

Trzecim i zarazem najbardziej kluczowym z perspektywy wdrożeniowej ograniczeniem jest zjawisko **przesunięcia dziedziny** (ang. *Domain Shift*) oraz podatność na dane z poza rozkładu. W celu weryfikacji zdolności uogólniania w trybie *Zero-Shot* na rzeczywistych danych medycznych, przeprowadzono test na publicznie dostępnym zbiorze DRIVE (ang. *Digital Retinal Images for Vessel Extraction*), zawierającym zdjęcia angiograficzne dna oka. Pomimo zastosowania standardowych technik wyrównywania kontrastu (CLAHE) oraz inwersji barw w fazie preprocessingu, model nie zdołał poprawnie zrekonstruować układu naczyń. Zestawienie tych predykcji zaprezentowano na Rysunku 5.8.



RYSUNEK 5.8: Ewaluacja w trybie *Zero-Shot* na fragmentach obrazów ze zbioru medycznego DRIVE. Widoczne całkowite załamanie procesu wnioskowania przestrzennego, objawiające się nadmierną aktywacją maski przezroczystości oraz wystąpieniem strukturalnych artefaktów szachownicy (ang. *grid artifacts*). Widoczny na wizualizacji termin *predykcja alpha* odnosi się do wyjściowego prawdopodobieństwa sieci ($p \in [0, 1]$), stanowiącego analityczną maskę obecności fizycznego obiektu w danej komórce.

Moduł percepcyjny, zoptymalizowany wyłącznie na syntetycznych obrazach o jednorodnym czarnym tle, napotkał dane spoza swojego rozkładu treningowego (ang. *Out-of-Distribution*). W rezultacie sieć nie potrafiła wyekstrahować użytecznych cech z zaszumionej, szarej tkanki anatomicznej. Brak znajomych wzorców doprowadził do załamania predykcji – zdeorientowany model aktywował maksymalne prawdopodobieństwo wystąpienia obiektu ($p \rightarrow 1.0$) dla niemal wszystkich komórek siatki, generując нефizyczne artefakty kratowe. Ten wynik empirycznie potwierdza, że architektura uczy się silnej i precyzyjnej reprezentacji dedykowanej dla określonego środowiska, jednak jest wysoce wrażliwa na zmianę domeny wizualnej i przed wdrożeniem klinicznym bezwzględnie będzie wymagać etapu strojenia wag (ang. *Fine-Tuning*).

Rozdział 6

Wnioski i kierunki dalszych badań

Głównym celem niniejszej pracy było zaprojektowanie i ewaluacja autoenkodera neurosymbolicznego, zdolnego do nienadzorowanej rekonstrukcji sekwencyjnych struktur wizualnych o zmiennej długości oraz złożonej topologii. Przeprowadzone eksperymenty oraz uzyskane wyniki pozwalają na sformułowanie następujących wniosków, stanowiących bezpośrednią odpowiedź na postawione we wstępie pytania badawcze:

- **Ad. P1 (Spójność fizyczna):** Zaproponowana funkcja straty, agregująca maskowane błędy trajektorii, nieciągłości, rozmiaru oraz problemu stopu, okazała się wysoce skuteczna. Oparcie procesu minimalizacji błędów na renderowaniu różniczkowalnym wymusiło na sieci generowanie struktur, które są spójne fizycznie i topologicznie, eliminując zjawisko "rozrywania" łańcuchów.
- **Ad. P2 (Mechanizmy modelowania):** Badania empiryczne wykazały przewagę architektury GRU nad LSTM w kontekście analizy prostych sekwencji. Mniejsza liczba parametrów komórki GRU, wsparta mechanizmem integracji globalnego kontekstu (ang. *context fusion*) oraz zastosowaniem przycinania gradientów (ang. *gradient clipping*), zapewniła stabilną konwergencję. Z kolei dla struktur o złożonej topologii (skrzyżowania i bifurkacje) udowodniono, że niezbędna jest zmiana paradygmatu na przestrzenną dekompozycję siatkową, która pozwala na równoległe wnioskowanie bez narzucania sztucznego rygoru sekwencyjnego.
- **Ad. P3 (Interpretowalność i manipulacja):** W przeciwieństwie do klasycznych modeli typu "czarna skrzynka", uzyskana reprezentacja jest w pełni interpretowalna. Model transformuje przestrzeń pikseli do postaci czytelnych parametrów geometrycznych (współrzędne, promienie, kolory). Pozwala to na bezpośrednią i bezbłędną ingerencję w wygenerowaną strukturę przed jej ostatecznym wyrenderowaniem, co spełnia kluczowy postulat paradygmatu neurosymbolicznego.
- **Ad. P4 (Wyuczalne parametry skalowania):** Udowodniono, że optymalizacja *end-to-end* globalnych mnożników promieni (S_{rx}, S_{ry}) całkowicie automatyzuje proces formowania topologii. Mechanizm ten pozwolił sieci na analityczne „zrozumienie” konieczności łączenia dyskretnych elips w ciągle, gładkie trajektorie, co z powodzeniem zastąpiło ręczny dobór hiperparametrów i uodporniło proces na początkowe błędy lokalizacyjne w fazie uczenia.

Dodatkowo, eksperymenty wykazały znaczną odporność zaprezentowanego systemu na stochastyczne degradacje obrazu wejściowego. Model skutecznie odtwarzał ciągłość struktury pomimo wysokiego poziomu szumu Gaussa, rozmycia optycznego oraz ubytków danych (ang. *dropout*), co potwierdzono za pomocą metryki IoU.

6.1 Wyzwania implementacyjne i analiza stabilności

Proces implementacji systemu neurosymbolicznego dostarczył szeregu wyzwań, które wymagały wyjścia poza standardowe schematy znane z literatury przedmiotu. Najistotniejszym z nich okazała się stabilizacja procesu uczenia dekodera rekurencyjnego. W początkowych iteracjach model wykazywał skłonność do „rozbiegania się” parametrów geometrycznych, co objawiało się generowaniem prymitywów o ujemnych promieniach lub wartościach wychodzących poza znormalizowany obszar $[0, 1]$.

Kluczowym momentem w pracach nad projektem było wprowadzenie mechanizmu wymuszania nauczyciela (ang. *teacher forcing*) z wygaszaniem liniowym. Zaobserwowano, że w fazie inicjalnej model sekwencyjny nie potrafi samodzielnie zainicjować łańcucha, ponieważ błąd generacji pierwszego elementu rzutuje na całą strukturę. Zastosowanie wymuszenia w pierwszych 40 epokach pozwoliło sieci na „nauczenie się” poprawnej składni geometrii, zanim przystąpiła do samodzielnego wnioskowania. Równie krytycznym parametrem okazał się współczynnik ostrości (s) w rendererze różniczkowalnym. Ustalenie jego wartości na $s = 20.0$ było wynikiem kompromisu między płynnością gradientu a precyzją krawędzi – zbyt wysokie wartości powodowały artefakty w propagacji wstecznej, uniemożliwiając konwergencję funkcji straty. Z kolei w przypadku architektury siatkowej, fundamentalne okazało się fizyczne oddzielenie nauki topologii (α_{mask}) od predykcji palety RGB w funkcji kosztu, co wyeliminowało problem utykania optymalizatora w lokalnych minimach.

6.2 Ewolucja podejścia i nieudane ścieżki badawcze

W trakcie prac badawczych nad modelem przetestowano szereg podejść, z których część posłużyła jako cenna lekcja pozwalająca na ostateczne udoskonalenie systemu. Opis tych etapów jest istotny z punktu widzenia metodyki prowadzenia badań.

6.2.1 Ewolucja podejścia do bifurkacji i ograniczenia modeli sekwencyjnych

Początkowe eksperymenty z rozszerzeniem modelu o obsługę topologii grafowych (skrzyżowania i rozgałęzienia) prowadzone wyłącznie w oparciu o architekturę rekurencyjną (GRU) zakończyły się niepowodzeniem. Zaimplementowany mechanizm powrotu (ang. *backtracking*) powodował drastyczną niestabilność sieci, która przy skrzyżowaniach typu X wpadała w nieskończone pętle i generowała nakładające się na siebie elementy.

Te fundamentalne trudności stały się bezpośrednią motywacją do opracowania opisanego w pracy przestrzennego dekodera siatkowego. Ostateczne eksperymenty udowodniły, że odejście od wymuszania sztucznego, sekwencyjnego porządku na korzyść równoległego wnioskowania przestrzennego całkowicie wyeliminowało problem wpadania w pętle, stanowiąc docelowe rozwiązanie dla struktur o złożonej topologii.

6.2.2 Podejście oparte na strukturach kolejkowych i BFS

W innej próbie stabilizacji architektur rekurencyjnych wykorzystano strukturę kolejki w procesie dekodowania, połączonej z algorytmem przeszukiwania wszcz (BFS). Zakładano, że generowanie obiektów w kolejności od środka na zewnątrz (warstwami) pozwoli na lepszą generalizację liczebności elementów. Eksperyment ten napotkał barierę w postaci konieczności stosowania maskowania (*paddingu*) dla niepełnych poziomów drzewa. Wprowadzone maski zakłócały gradient w module rekurencyjnym, powodując, że model „zapominał” o strukturze całego łańcucha na rzecz lokalnego dopasowania do bieżącego poziomu kolejki.

6.3 Kierunki dalszych badań

Zaproponowana i przetestowana architektura hybrydowa stanowi stabilny fundament, jednak specyfika danych wizualnych otwiera szerokie pole do dalszych optymalizacji. Planowane prace rozwojowe obejmują:

1. **Nienadzorowana adaptacja domenowa (UDA) i medyczny transfer wiedzy:** Jak wykazano w analizie przypadków brzegowych, wdrożenie na zbiory rzeczywiste (takie jak DRIVE) skutkuje problemem przesunięcia dziedziny (ang. *Domain Shift*). Planuje się wdrożenie technik adaptacji domenowej z wykorzystaniem uczenia kontrastowego (ang. *Contrastive Learning*) lub architektur typu CycleGAN. Pozwoli to na adaptację modelu do śledzenia układu naczyń krwionośnych na rzeczywistych obrazach ultrasonograficznych (USG) oraz w angiografii, umożliwiając implementację w środowisku klinicznym bez konieczności kosztownego, ręcznego etykietowania.
2. **Zastosowanie architektury Transformer:** Ponieważ modele oparte na gęstych siatkach skutecznie omijają zjawisko zagłódzenia przestrzeni ukrytej, naturalnym rozwinięciem jest zastosowanie mechanizmów samouwagi (ang. *Self-Attention*). Przejście na architekturę Transformer (inspirowaną m.in. modelem DETR) pozwoliłoby zrezygnować z restrykcyjnego podziału na komórki macierzy na rzecz bezpośredniego, równoległego i bezkolizyjnego przewidywania nieuporządkowanych zbiorów geometrycznych.
3. **Ekspansja reprezentacji do przestrzeni trójwymiarowej (3D):** Zastosowany paradygmat *analysis-by-synthesis* można rozszerzyć o wymiar głębi, modyfikując renderer różniczkowalny do operowania na sferoidach i woksela. Pozwoliłoby to na rekonstrukcję wolumetrycznych sieci naczyniowych bezpośrednio ze skanów tomografii komputerowej (TK) lub rezonansu magnetycznego (MRI), oferując w pełni edytowalne modele 3D.
4. **Rozszerzenie w wymiar czasu (Analiza Wideo):** Implementacja w analitycznym rendererze zmiennej czasowej t , co pozwoliłoby na śledzenie dynamiki przepływu lub zmian morfologicznych układów (np. pulsujących naczyń krwionośnych) w ciągłych sekwencjach wideo, przy jednoczesnym utrzymaniu czasu inferencji w granicach systemów czasu rzeczywistego.

6.4 Autorefleksja metodologiczna i zagrożenia

Zastosowane podejście, choć wysoce efektywne w modelowaniu obiektów syntetycznych, posiada wrodzone ograniczenia. Największym zagrożeniem jest zjawisko biasu w kierunku danych treningowych – model jest „rozpieszczony” idealnymi, geometrycznymi kształtami z generatora. W warunkach rzeczywistych, gdzie granice obiektów są nieostre, a oświetlenie niejednorodne, proponowany enkoder może wymagać znacznie większej pojemności informacyjnej.

Koszt obliczeniowy renderowania sekwencyjnego rośnie liniowo wraz z długością sekwencji, natomiast podejście siatkowe narzuca stały narzut przestrzenny komórek. Przy konieczności zaimplementowania siatki o skrajnie dużej rozdzielczości (np. dla obrazów 4K) mogłoby to stanowić barierę alokacyjną dla pamięci VRAM.

Mimo to, doskonałe wyniki IoU uzyskane w pracy oraz pokonanie ograniczeń topologicznych potwierdzają, że neurosymboliczna dekompozycja obrazu w trybie *analysis-by-synthesis* jest skuteczną i bardzo obiecującą drogą dla interpretowalnej sztucznej inteligencji.

Mimo wskazanych ograniczeń, zaproponowane w niniejszej pracy rozwiązanie stanowi istotny krok w kierunku budowy w pełni interpretowalnych systemów sztucznej inteligencji. Otwarte udo-

stępnienie kodu źródłowego oraz zaimplementowanego generatora danych syntetycznych sprzyja transparentności, umożliwiając łatwą weryfikację, replikację wyników oraz dalszą rozbudowę zaprezentowanych architektur neurosymbolicznych przez szerszą społeczność badawczą.

Literatura

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553), 2015.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
<http://www.deeplearningbook.org>.
- [3] Artur S. d'Avila Garcez et al. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *arXiv preprint arXiv:1905.06088*, 2019.
- [4] Tarek R. Besold et al. Neural-symbolic learning and reasoning: contributions and challenges. In *Papers from the 2015 AAAI Spring Symposium / No. 3: Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches*, 2015.
- [5] S. M. Ali Eslami et al. Attend, infer, repeat: Fast scene understanding with generative models. *arXiv preprint arXiv:1603.08575*, 2016.
- [6] Klaus Greff et al. Binding via reconstruction clustering. *arXiv preprint arXiv:1511.06418*, 2015.
- [7] Peter Battaglia et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- [8] David Ha and Douglas Eck. A neural representation of sketch drawings. *arXiv preprint arXiv:1704.03477*, 2017.
- [9] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [10] Michał Falbogowski. Compositional representation learning via reconstruction, 2023. Master's thesis, Poznań University of Technology.
- [11] Tzu-Mao Li et al. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6), 2020.
- [12] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.
- [13] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.
- [14] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. In *Science*, volume 313, 2006.
- [15] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2), 1994.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8), 1997.
- [17] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

- [18] Zuzanna Buchnajzer, Kacper Dobek, Stanisław Hapke, and Daniel Jankowski. Deep learning for compositional geometric modeling in medical imaging, 2024. Bachelor's thesis, Poznań University of Technology.
- [19] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *arXiv preprint arXiv:1409.0473*, 2014.
- [20] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [21] Christopher P. Burgess et al. Understanding disentangling in beta-vae. In *NeurIPS Workshop on Learning Disentangled Representations*, 2018.
- [22] Christopher P Burgess, Loic Matthey, Nicholas Watters, Rishabh Kabra, Irina Higgins, Matt Botvinick, and Alexander Lerchner. Monet: Unsupervised scene decomposition and representation. *arXiv preprint arXiv:1901.11390*, 2019.
- [23] Klaus Greff, Raphael Lopez Kaufman, Rishabh Kabra, Nick Watters, Christopher Burgess, Daniel Zoran, Loic Matthey, Matthew Botvinick, and Alexander Lerchner. Multi-object representation learning with iterative variational inference. In *International Conference on Machine Learning*, pages 2424–2433. PMLR, 2019.
- [24] Zhixuan Lin et al. Space: Unsupervised object-oriented scene representation via spatial attention and decomposition. In *International Conference on Learning Representations*, 2020.
- [25] Shichen Liu et al. Soft rasterizer: A differentiable renderer for image-based 3d reasoning. *arXiv preprint arXiv:1904.01786*, 2019.
- [26] Hiroharu Kato, Yoshitaka Ushiku, and Tatsuya Harada. Neural 3d mesh renderer. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3907–3916, 2018.
- [27] Jiayuan Mao, Chuang Gan, Pushmeet Kohli, Joshua B Tenenbaum, and Jiajun Wu. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. In *International Conference on Learning Representations*, 2019.
- [28] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 835–850, 2021.
- [29] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [30] Yann LeCun et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 1998.
- [31] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, University of Toronto, Toronto, Ontario, 2009.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [33] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

- [34] John D Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [35] Alex Clark et al. *Pillow (PIL Fork) Documentation*, 2026. <https://python-pillow.org/>.
- [36] Gary Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 25(11):120–123, 2000.
- [37] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [38] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [39] James Lucas, George Tucker, Roger Grosse, and Mohammad Norouzi. Understanding posterior collapse in generative latent variable models. *Deep Learning and Information-Theoretic Workshop at ICLR*, 2019.

Dodatek A: Kod źródłowy i odtwarzanie eksperymentów

Kod źródłowy zaimplementowanego systemu neurosymbolicznego, włączając w to generator danych syntetycznych oraz wszystkie warianty architektoniczne modułu wnioskowania, został udostępniony w publicznym repozytorium GitHub¹.

Aby pobrać kod projektu na lokalną maszynę i zainstalować wymagane zależności, należy zainstalować oprogramowanie Git i wykonać poniższe polecenia w terminalu:

```
$ REP_NAME="Neurosymbolic-Autoencoders-for-Modelling-of-2D-Structures"
$ git clone https://github.com/pchumski/$REP_NAME
$ cd $REP_NAME
$ pip install -r requirements.txt
```

Generowanie danych w projekcie odbywa się dynamicznie (w locie) podczas procesu uczenia za pomocą autorskiego, wbudowanego generatora parametrycznego. Dzięki temu repozytorium jest całkowicie samowystarczalne i nie wymaga pobierania zewnętrznych, wielkogabarytowych zbiorów danych.

Skrypt główny wykonujący wszystkie zaprezentowane w pracy eksperymenty znajduje się w głównym katalogu pod nazwą `main.py`. Szczegółowy opis parametrów konfiguracyjnych można uzyskać wywołując następującą komendę:

```
$ python main.py --help
```

Po wywołaniu powyższego polecenia wyświetlony zostanie następujący komunikat (część opcji pominięto dla zachowania czytelności):

```
usage: main.py [-h] [--model-type {gru,grid}] ...
```

options:

```
-h, --help           Wyświetla ten komunikat pomocy i kończy działanie.
--model-type {gru,grid}
                     Wybór wariantu architektonicznego dekodera.
--dataset-type {geometric,bifurcation}
                     Wybór generatora danych (proste węże vs rozgałęzienia).
--grid-size GRID_SIZE
                     Rozmiar siatki przestrzennej (domyślnie 16).
--noise-severity SEVERITY
                     Poziom degradacji obrazu wejściowego (0.0 - 1.0).
--epochs EPOCHS      Liczba epok treningowych.
--batch-size BATCH_SIZE
                     Rozmiar paczki danych (batch size).
--lr LR              Współczynnik uczenia (learning rate).
--device {cpu,cuda}   Wybór akceleratora obliczeniowego.
```

Aby odtworzyć jeden z kluczowych eksperymentów (np. optymalizację przestrzennego dekodera siatkowego na danych z ekstremalnym szumem), należy uruchomić poniższe polecenie w wierszu poleceń. Skrypt ten wygeneruje na bieżąco strukturę grafową, zainicjuje model z siatką 16×16 i przeprowadzi trening przez 60 epok z wykorzystaniem akceleracji GPU:

¹<https://github.com/pchumski/Neurosymbolic-Autoencoders-for-Modelling-of-2D-Structures>

```
$ python main.py \  
  --device=cuda \  
  --model-type=grid \  
  --dataset-type=geometric \  
  --grid-size=16 \  
  --epochs=60 \  
  --batch-size=32 \  
  --lr=2e-4 \  
  --noise-severity=0.7
```



© 2026 Paweł Chumski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.